SID4: 1066

PORT_BASE: 8166

PREFIX: s1066

SEED: 1066

VERIFY_SEED: 261066

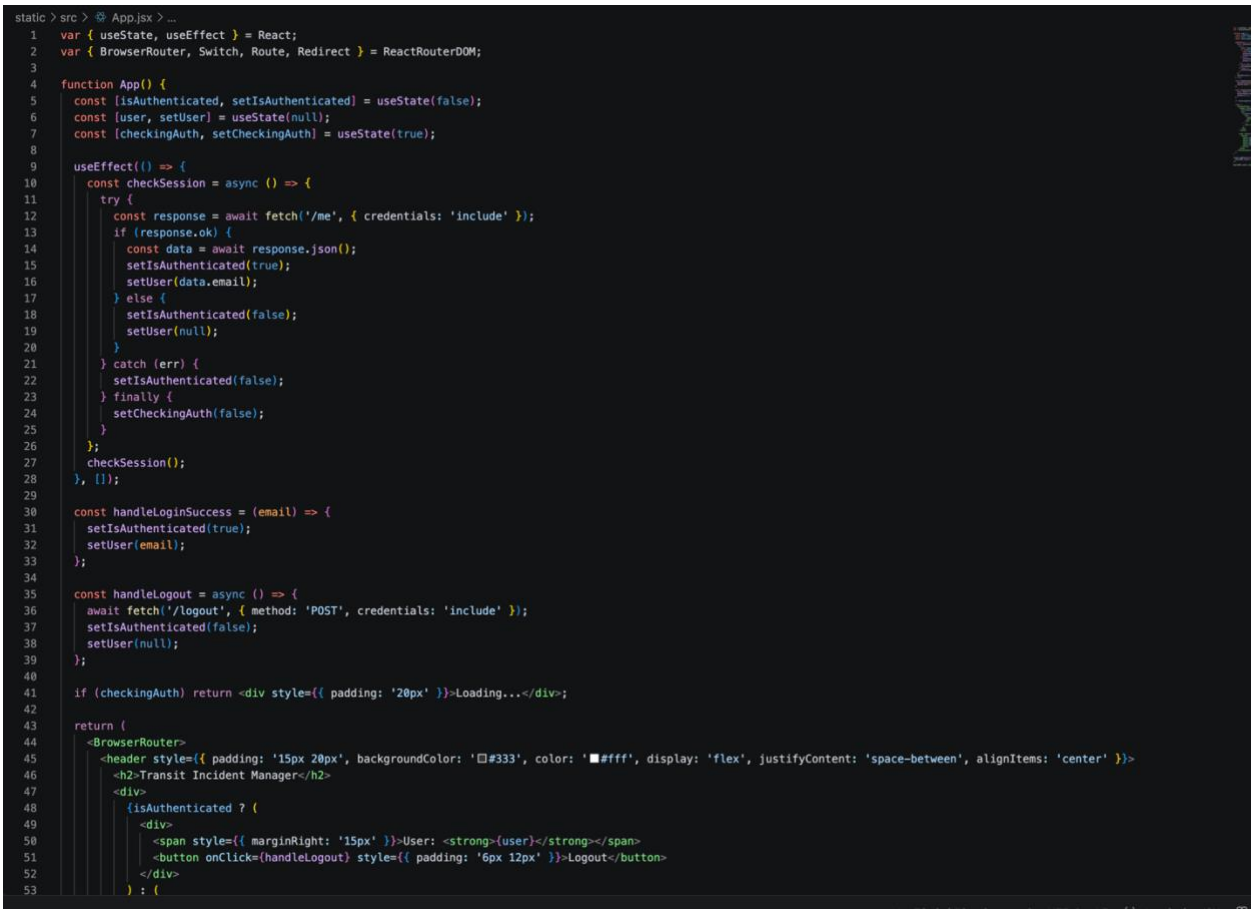
DOMAIN_ID: 2

Hardware: Macintosh HD, Apple M4, 24 GB, Tahoe 26.6.2

Tagged Commit Hash: bcdfa45f1c615aa4fd5a1fba7ad4c83f0ae682c1

https://github.com/paknathan/data260-1066

Part 1)



```
          ) : (
            <span style={{ color: '#aaa' }}>Not Authenticated</span>
          )}
        </div>
      </header>

      <main style={{ padding: '20px', maxWidth: '800px', margin: '0 auto' }}>
        <Switch>
          <Route exact path="/" render={() => <Home isAuthenticated={isAuthenticated} />} />
          <Route path="/login" render={() => (isAuthenticated ? <Redirect to="/" /> : <Login onLoginSuccess={handleLoginSuccess} />)} />
          <Route path="/create" render={() => (isAuthenticated ? <CreateRecord /> : <Redirect to="/login" />)} />
          <Route path="/update" render={() => (isAuthenticated ? <UpdateRecord /> : <Redirect to="/login" />)} />
          <Route path="/delete" render={() => (isAuthenticated ? <DeleteRecord /> : <Redirect to="/login" />)} />
        </Switch>
      </main>
    </BrowserRouter>
  );
}

window.addEventListener('load', () => {
  ReactDOM.render(<App />, document.getElementById('root'));
});

ReactDOM.render(<App />, document.getElementById('root'));
```

static > src > components > Login.jsx > ...

```
var { useState } = React;
var { useHistory } = ReactRouterDOM;

function Login({ onLoginSuccess }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const history = useHistory();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    try {
      const response = await fetch('/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({ email, password }),
      });

      if (!response.ok) throw new Error('Invalid email or password.');

      const data = await response.json();
      onLoginSuccess(data.email);
      history.push('/');
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div style={{ maxWidth: '400px', margin: '40px auto', padding: '20px', border: '1px solid #ccc', borderRadius: '6px' }}>
      <h3>Sign In</h3>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block', marginBottom: '5px' }}>Email</label>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            placeholder="admin@transit.org"
            required
            style={{ width: '100%', padding: '8px', boxSizing: 'border-box' }}
          />
        </div>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block', marginBottom: '5px' }}>Password</label>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
```

```
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            style={{ width: '100%', padding: '8px', boxSizing: 'border-box' }}
          />
        </div>
        <button type="submit" style={{ padding: '8px 16px', backgroundColor: '#007bff', color: '#fff', border: 'none', cursor: 'pointer' }}>
          Log In
        </button>
      </form>
    </div>
  );
}

window.Login = Login;
```

static > src > components > Home.jsx > ...

```
var { useState, useEffect } = React;
var { Link } = ReactRouterDOM;

function Home({ isAuthenticated }) {
  const [records, setRecords] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    if (isAuthenticated) {
      fetchRecords();
    } else {
      setLoading(false);
    }
  }, [isAuthenticated]);

  const fetchRecords = async () => {
    try {
      const response = await fetch('/incidents', { credentials: 'include' });
      if (!response.ok) throw new Error('Failed to fetch records.');
      const data = await response.json();
      setRecords(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  if (!isAuthenticated) {
    return (
      <div style={{ textAlign: 'center', marginTop: '50px' }}>
        <h2>Login required</h2>
        <p>Please log in to view transit incident records and perform CRUD operations.</p>
        <Link to="/login" style={{ color: '#007bff', fontWeight: 'bold' }}>Go to Login</Link>
      </div>
    );
  }

  return (
    <div>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '20px' }}>
        <h3>Municipal Transit Incidents</h3>
        <div>
          <Link to="/create" style={{ padding: '8px 12px', backgroundColor: '#28a745', color: '#fff', textDecoration: 'none', borderRadius: '4px', marginRight: '8px' }}>+ Add Record</Link>
          <Link to="/update" style={{ padding: '8px 12px', backgroundColor: '#ffc107', color: '#000', textDecoration: 'none', borderRadius: '4px', marginRight: '8px' }}>Update Record</Link>
          <Link to="/delete" style={{ padding: '8px 12px', backgroundColor: '#dc3545', color: '#fff', textDecoration: 'none', borderRadius: '4px' }}>Delete Record</Link>
        </div>
      </div>

      {loading && <p>Loading records...</p>}
      {error && <p style={{ color: 'red' }}>{error}</p>}

```

```
      {loading && <p>Loading records...</p>}
      {error && <p style={{ color: 'red' }}>{error}</p>}

      {!loading && !error && (
        <table border="1" cellPadding="10" cellSpacing="0" style={{ width: '100%', borderCollapse: 'collapse' }}>
          <thead>
            <tr style={{ backgroundColor: '#f2f2f2' }}>
              <th>Incident ID</th>
              <th>Route / Line (Primary)</th>
              <th>Incident Type (Secondary)</th>
              <th>Description</th>
            </tr>
          </thead>
          <tbody>
            {records.length > 0 ? (
              records.map((item) => (
                <tr key={item.incident_id}>
                  <td>{item.incident_id}</td>
                  <td>{item.route_or_line}</td>
                  <td>{item.incident_type}</td>
                  <td>{item.description}</td>
                </tr>
              ))
            ) : (
              <tr>
                <td colSpan="4" style={{ textAlign: 'center' }}>No incident records found.</td>
              </tr>
            )}
          </tbody>
        </table>
      )}
    </div>
  );
}

window.Home = Home;
```

static > src > components > CreateRecord.jsx > CreateRecord > handleSubmit > response

```
var { useState } = React;
var { useHistory } = ReactRouterDOM;

function CreateRecord() {
  const [routeOrLine, setRouteOrLine] = useState('');
  const [incidentType, setIncidentType] = useState('');
  const [description, setDescription] = useState('');
  const [error, setError] = useState('');
  const history = useHistory();

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch('/incidents', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({ route_or_line: routeOrLine, incident_type: incidentType, description }),
      });
      if (!response.ok) throw new Error('Failed to create record.');
      history.push('/');
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div style={{ maxWidth: '500px', margin: '0 auto' }}>
      <h3>Add New Incident Record</h3>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>Route or Line (Primary Field):</label>
          <input type="text" value={routeOrLine} onChange={(e) => setRouteOrLine(e.target.value)} required style={{ width: '100%', padding: '8px' }} />
        </div>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>Incident Type (Secondary Field):</label>
          <input type="text" value={incidentType} onChange={(e) => setIncidentType(e.target.value)} required style={{ width: '100%', padding: '8px' }} />
        </div>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>Description:</label>
          <textarea value={description} onChange={(e) => setDescription(e.target.value)} style={{ width: '100%', padding: '8px' }} />
        </div>
        <button type="submit" style={{ padding: '8px 16px', backgroundColor: '#28a745', color: '#fff', border: 'none' }}>Add Record</button>
      </form>
    </div>
  );
}

window.CreateRecord = CreateRecord;
```

static > src > components > UpdateRecord.jsx > ...

```jsx
var { useState } = React;
var { useHistory } = ReactRouterDOM;

function UpdateRecord() {
  const [incidentId, setIncidentId] = useState('');
  const [routeOrLine, setRouteOrLine] = useState('');
  const [incidentType, setIncidentType] = useState('');
  const [error, setError] = useState('');
  const history = useHistory();

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch(`/incidents/${incidentId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({ route_or_line: routeOrLine, incident_type: incidentType }),
      });
      if (!response.ok) throw new Error('Failed to update record.');
      history.push('/');
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div style={{ maxWidth: '500px', margin: '0 auto' }}>
      <h3>Update Record</h3>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>Incident ID to Update:</label>
          <input type="text" value={incidentId} onChange={(e) => setIncidentId(e.target.value)} required placeholder="e.g. INC-00147" style={{ width: '100%', padding: '8px' }} />
        </div>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>New Route / Line:</label>
          <input type="text" value={routeOrLine} onChange={(e) => setRouteOrLine(e.target.value)} required style={{ width: '100%', padding: '8px' }} />
        </div>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>New Incident Type:</label>
          <input type="text" value={incidentType} onChange={(e) => setIncidentType(e.target.value)} required style={{ width: '100%', padding: '8px' }} />
        </div>
        <button type="submit" style={{ padding: '8px 16px', backgroundColor: '#ffc107', border: 'none' }}>Update Record</button>
      </form>
    </div>
  );
}

window.UpdateRecord = UpdateRecord;
```

static > src > components > DeleteRecord.jsx > ...

```jsx
var { useState } = React;
var { useHistory } = ReactRouterDOM;

function DeleteRecord() {
  const [incidentId, setIncidentId] = useState('');
  const [error, se var useHistory: any
  const history = useHistory();

  const handleDelete = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch(`/incidents/${incidentId}`, {
        method: 'DELETE',
        credentials: 'include',
      });
      if (!response.ok) throw new Error('Failed to delete record.');
      history.push('/');
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div style={{ maxWidth: '500px', margin: '0 auto' }}>
      <h3>Delete Record</h3>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <form onSubmit={handleDelete}>
        <div style={{ marginBottom: '15px' }}>
          <label style={{ display: 'block' }}>Incident ID to Delete:</label>
          <input type="text" value={incidentId} onChange={(e) => setIncidentId(e.target.value)} required placeholder="e.g. INC-00147" style={{ width: '100%', padding: '8px' }} />
        </div>
        <button type="submit" style={{ padding: '8px 16px', backgroundColor: '#dc3545', color: '#fff', border: 'none' }}>Delete Record</button>
      </form>
    </div>
  );
}

window.DeleteRecord = DeleteRecord;
```

main.py > ...

```python
# main.py
import uvicorn
from fastapi import FastAPI, Request, HTTPException, status
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from starlette.middleware.sessions import SessionMiddleware
from routers.auth import router

app = FastAPI(title="Municipal Transit Incident API")

# Enable Session Middleware for HTTP-only cookies
app.add_middleware(
    SessionMiddleware,
    secret_key="data260-session-secret",
    https_only=True,
    same_site="lax",
    max_age=3600
)

app.include_router(router)

# In-memory database (will move to MySQL in Part 2)
incidents_db = [
    {
        "incident_id": "INC-00147",
        "route_or_line": "Route 22",
        "incident_type": "Mechanical Breakdown",
        "email": "operator@transit.org",
        "description": "Engine failure near 4th street stop."
    }
]

# CRUD API ENDPOINTS
@app.get("/incidents")
async def get_incidents(request: Request):
    user = request.session.get("user")
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Login required")
    return incidents_db

@app.post("/incidents")
async def create_incident(request: Request):
    user = request.session.get("user")
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Login required")

    data = await request.json()
    new_id = f"INC-{len(incidents_db) + 148:05d}"
    data["incident_id"] = new_id
    incidents_db.append(data)
    return data

```

```python

@app.put("/incidents/{incident_id}")
async def update_incident(incident_id: str, request: Request):
    user = request.session.get("user")
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Login required")

    data = await request.json()
    for item in incidents_db:
        if item["incident_id"] == incident_id:
            item["route_or_line"] = data.get("route_or_line", item["route_or_line"])
            item["incident_type"] = data.get("incident_type", item["incident_type"])
            return item

    raise HTTPException(status_code=404, det
                        (parameter) request: Request
                        request: Request

@app.delete("/incidents/{incident_id}")
async def delete_incident(incident_id: str, request: Request):
    user = request.session.get("user")
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Login required")

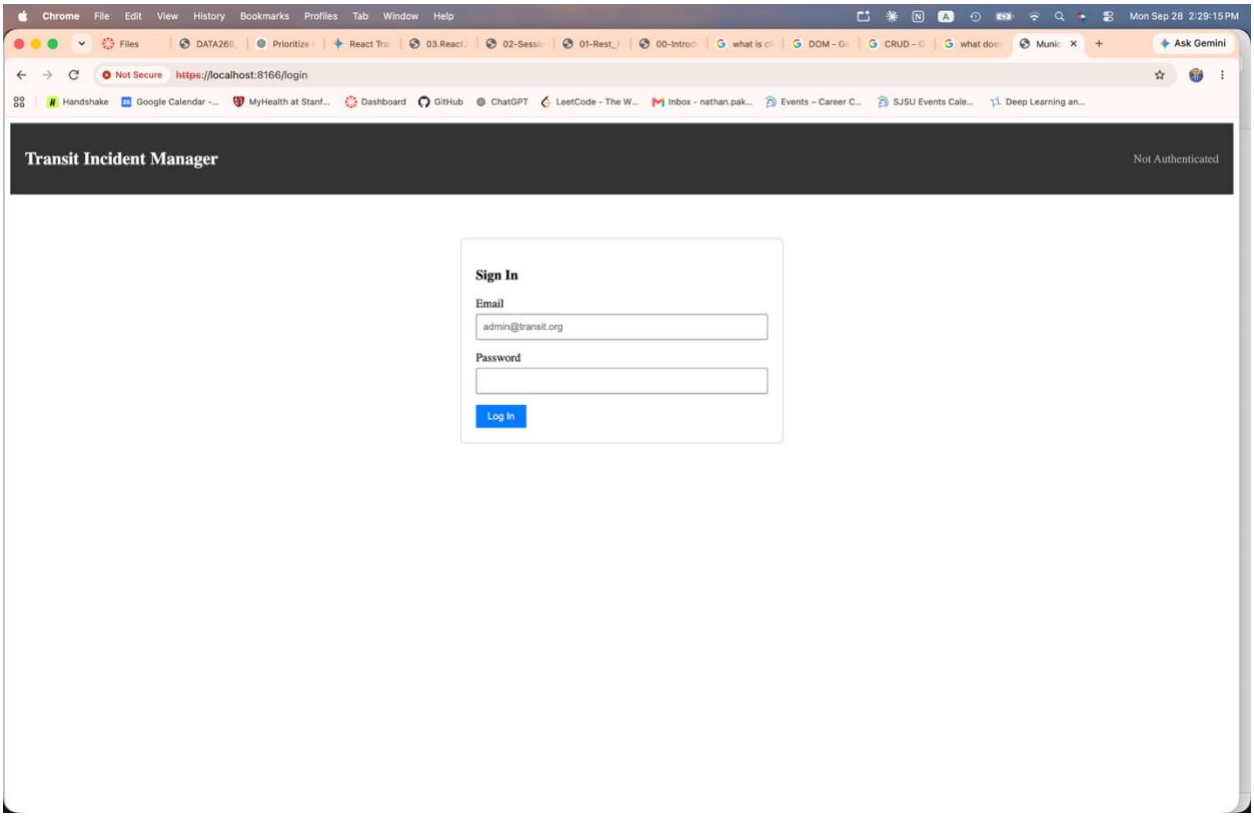
    global incidents_db
    incidents_db = [item for item in incidents_db if item["incident_id"] != incident_id]
    return {"message": "Record deleted"}

# Mount static files
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

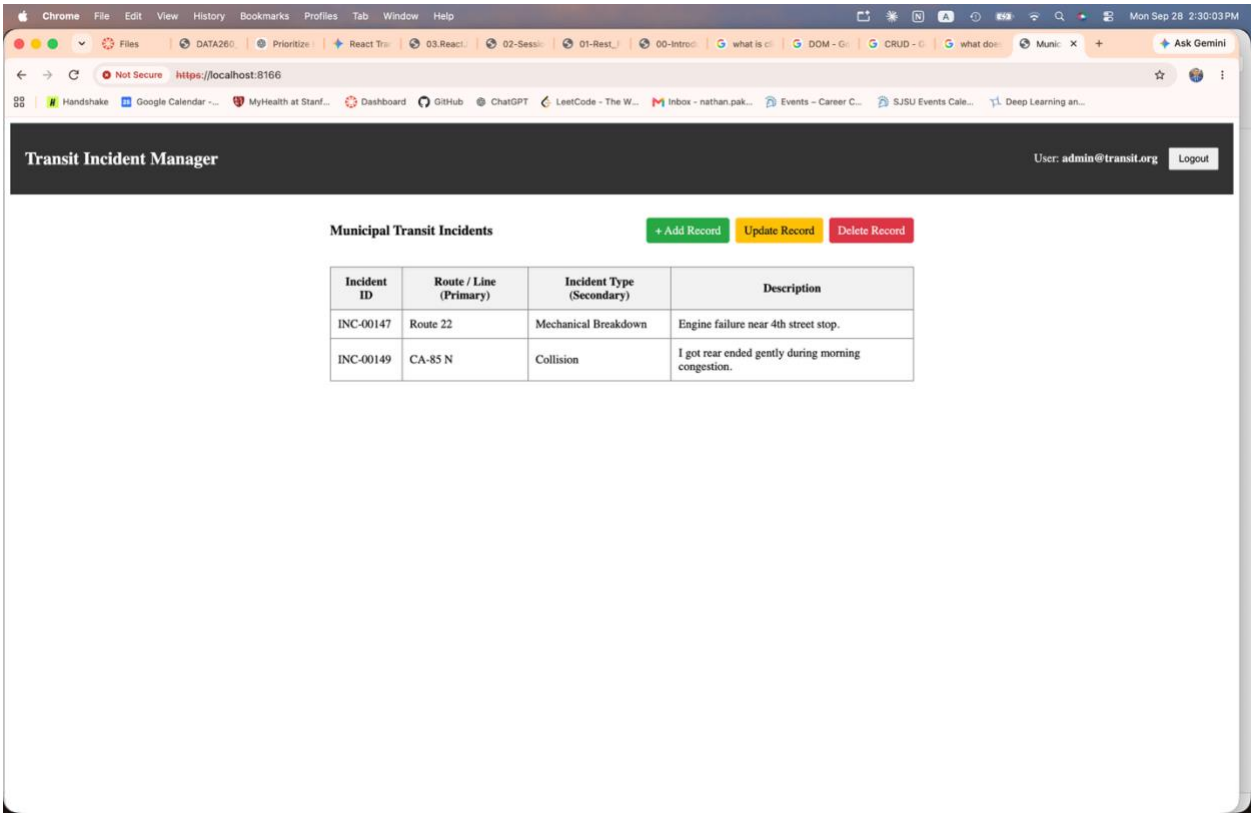
# React Catch-All Route: Serves index.html for all React Router paths
@app.get("/{full_path:path}")
def serve_react(request: Request, full_path: str):
    return templates.TemplateResponse("index.html", {"request": request})

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True, ssl_keyfile="key.pem", ssl_certfile="cert.pem")
```

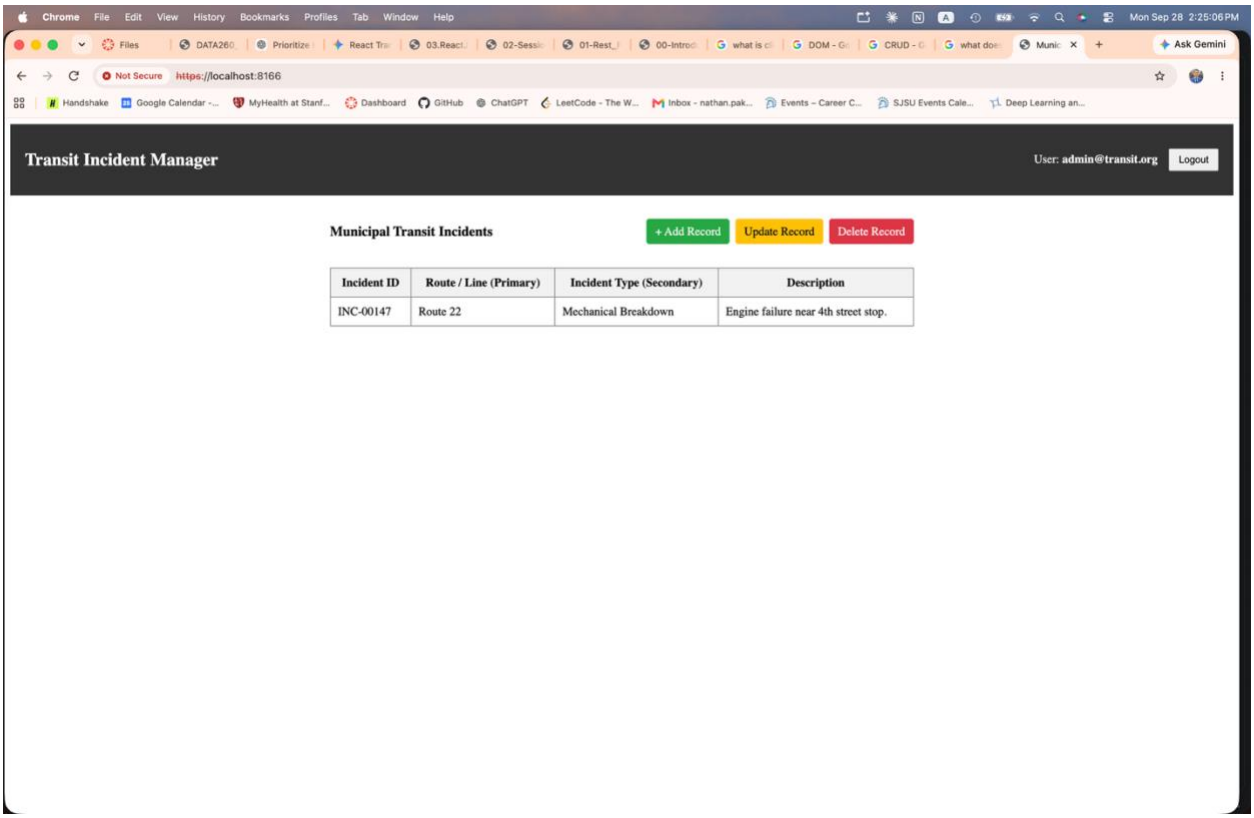
Logging In:



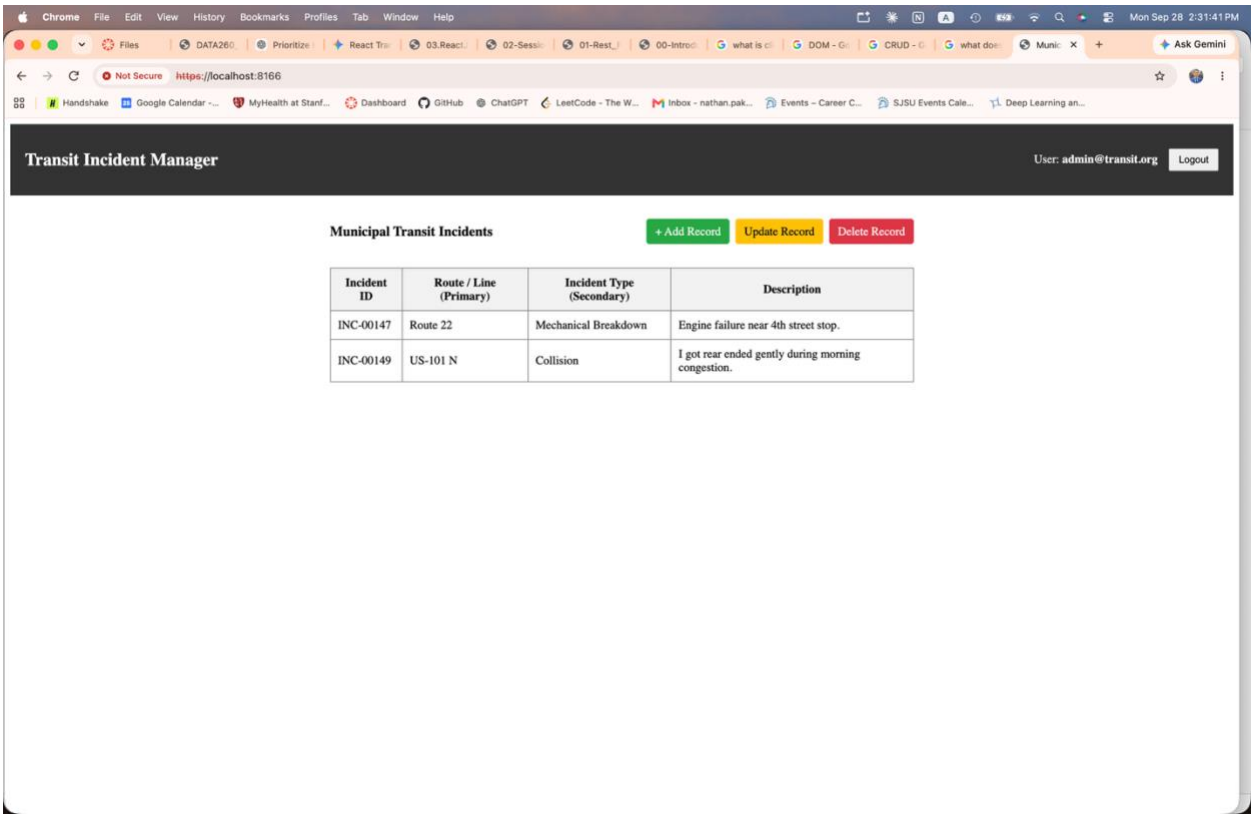
Adding an incident:



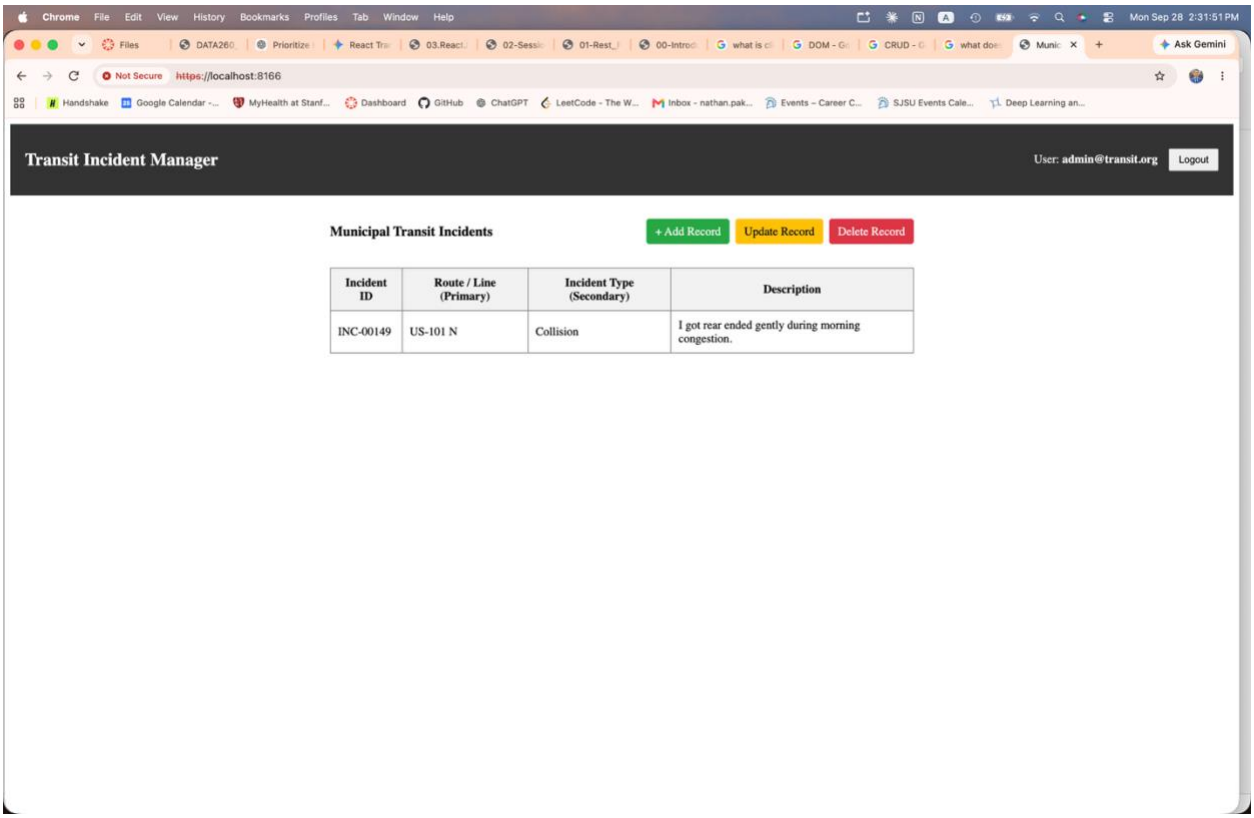
Home Page/Original List:



Updating Incident:



Deleting an Incident:



Part 2)

main.py > ...

```python
import os
import uuid
from datetime import datetime, timedelta
from typing import List, Optional

from fastapi import FastAPI, Depends, HTTPException, status, Response, Request
from fastapi.staticfiles import StaticFiles
from fastapi.responses import HTMLResponse, FileResponse
from pydantic import BaseModel, EmailStr
import bcrypt
from sqlalchemy import create_engine, Column, Integer, String, Text, DateTime, ForeignKey
from sqlalchemy.orm import declarative_base, sessionmaker, Session, relationship

# ==========================================================================
# 1. Database Configuration & Models (Database: Municipal_Rel)
# ==========================================================================
# Update host, user, and password as per your local MySQL setup
# Reads from terminal environment variable; falls back or raises an error if missing
DATABASE_URL = os.getenv("DATABASE_URL")

if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL environment variable is not set. Run `export DATABASE_URL=...` first.")

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# User Table
class UserModel(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(100), nullable=False)
    email = Column(String(120), unique=True, index=True, nullable=False)
    password_hash = Column(String(255), nullable=False)

    sessions = relationship("SessionModel", back_populates="user", cascade="all, delete-orphan")

# Server-Side Sessions Table
class SessionModel(Base):
    __tablename__ = "sessions"

    id = Column(String(64), primary_key=True, index=True)  # Opaque Session Token (UUID)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
    expires_at = Column(DateTime, nullable=False)

    user = relationship("UserModel", back_populates="sessions")

# Primary Domain Entity Table (Incident)
class IncidentModel(Base):
    __tablename__ = "incidents"
```

```python
# Primary Domain Entity Table (Incident)
class IncidentModel(Base):
    __tablename__ = "incidents"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    route_or_line = Column(String(100), nullable=False)  # Primary field
    incident_type = Column(String(100), nullable=False)  # Secondary field
    description = Column(Text, nullable=True)

# Create tables in MySQL if they do not exist
Base.metadata.create_all(bind=engine)

# ------------------------------------------------------------------------------
# 2. Database Dependency (Strictly using db_session_basede26)
# ------------------------------------------------------------------------------
def get_db():
    db_session_basede26 = SessionLocal()
    try:
        yield db_session_basede26
    finally:
        db_session_basede26.close()

# ------------------------------------------------------------------------------
# 3. Pydantic Schemas
# ------------------------------------------------------------------------------
class UserCreate(BaseModel):
    name: str
    email: EmailStr
    password: str

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class IncidentCreate(BaseModel):
    route_or_line: str
    incident_type: str
    description: Optional[str] = None

class IncidentUpdate(BaseModel):
    route_or_line: Optional[str] = None
    incident_type: Optional[str] = None
    description: Optional[str] = None

class IncidentResponse(BaseModel):
    incident_id: int
    route_or_line: str
    incident_type: str
    description: Optional[str] = None

    class Config:
        from_attributes = True
```

```python
    class Config:
Changed lines
        from_attributes = True

# ------------------------------------------------------------------------------
# 4. FastAPI Setup
# ------------------------------------------------------------------------------
app = FastAPI(title="Municipal Transit Incident Manager")

# Mount static files
app.mount("/static", StaticFiles(directory="static"), name="static")

# ------------------------------------------------------------------------------
# 5. Session Authentication Helper
# ------------------------------------------------------------------------------
def get_current_user(request: Request, db_session_basede26: Session = Depends(get_db)) -> UserModel:
    session_token = request.cookies.get("session_id")
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Not authenticated")

    session_record = db_session_basede26.query(SessionModel).filter(
        SessionModel.id == session_token
    ).first()

    if not session_record:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid session")

    if datetime.~~utcnow~~() > session_record.expires_at:
        db_session_basede26.delete(session_record)
        db_session_basede26.commit()
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Session expired")

    return session_record.user

# ------------------------------------------------------------------------------
# 6. Auth Endpoints
# ------------------------------------------------------------------------------
@app.post("/register", status_code=status.HTTP_210_CREATED if hasattr(status, 'HTTP_210_CREATED') else 201)
def register(user_data: UserCreate, db_session_basede26: Session = Depends(get_db)):
    existing_user = db_session_basede26.query(UserModel).filter(UserModel.email == user_data.email).first()
    if existing_user:
        raise HTTPException(status_code=400, detail="Email already registered")

    salt = bcrypt.gensalt()
    hashed_pw = bcrypt.hashpw(user_data.password.encode('utf-8'), salt).decode('utf-8')

    new_user = UserModel(
        name=user_data.name,
        email=user_data.email,
        password_hash=hashed_pw
    )
    db_session_basede26.add(new_user)
```

```python
    db_session_basede26.add(new_user)
Changed lines
    db_session_basede26.commit()
    return {"message": "User registered successfully"}

@app.post("/login")
def login(login_data: UserLogin, response: Response, db_session_basede26: Session = Depends(get_db)):
    user = db_session_basede26.query(UserModel).filter(UserModel.email == login_data.email).first()
    if not user or not bcrypt.checkpw(login_data.password.encode('utf-8'), user.password_hash.encode('utf-8')):
        raise HTTPException(status_code=401, detail="Invalid email or password")

    # Generate opaque server-side session token
    session_token = str(uuid.uuid4())
    created_at = datetime.utcnow()
    expires_at = created_at + timedelta(hours=8)

    session_record = SessionModel(
        id=session_token,
        user_id=user.id,
        created_at=created_at,
        expires_at=expires_at
    )
    db_session_basede26.add(session_record)
    db_session_basede26.commit()

    # Set HTTP-Only opaque cookie
    response.set_cookie(
        key="session_id",
        value=session_token,
        httponly=True,
        samesite="lax",
        secure=False  # Set True in HTTPS production
    )
    return {"email": user.email, "name": user.name}

@app.post("/logout")
def logout(request: Request, response: Response, db_session_basede26: Session = Depends(get_db)):
    session_token = request.cookies.get("session_id")
    if session_token:
        db_session_basede26.query(SessionModel).filter(SessionModel.id == session_token).delete()
        db_session_basede26.commit()

    response.delete_cookie("session_id")
    return {"message": "Logged out successfully"}

@app.get("/me")
def get_me(current_user: UserModel = Depends(get_current_user)):
    return {"email": current_user.email, "name": current_user.name}

# ------------------------------------------------------------------------
# 7. Domain Entity CRUD Endpoints (POST, GET all, GET by ID, PUT, DELETE)
# ------------------------------------------------------------------------
```

```python
# ------------------------------------------------------------------------

# 1. Add a new record (POST)
@app.post("/incidents", response_model=IncidentResponse, status_code=status.HTTP_201_CREATED)
def create_incident(
    data: IncidentCreate,
    current_user: UserModel = Depends(get_current_user),
    db_session_basede26: Session = Depends(get_db)
):
    incident = IncidentModel(
        route_or_line=data.route_or_line,
        incident_type=data.incident_type,
        description=data.description
    )
    db_session_basede26.add(incident)
    db_session_basede26.commit()
    db_session_basede26.refresh(incident)
    return IncidentResponse(
        incident_id=incident.id,
        route_or_line=incident.route_or_line,
        incident_type=incident.incident_type,
        description=incident.description
    )

# 2. View all records (GET)
@app.get("/incidents", response_model=List[IncidentResponse])
def get_all_incidents(
    current_user: UserModel = Depends(get_current_user),
    db_session_basede26: Session = Depends(get_db)
):
    records = db_session_basede26.query(IncidentModel).all()
    return [
        IncidentResponse(
            incident_id=item.id,
            route_or_line=item.route_or_line,
            incident_type=item.incident_type,
            description=item.description
        ) for item in records
    ]

@app.get("/incidents/{incident_id}", response_model=IncidentResponse)
def get_incident_by_id(
    incident_id: int,
    current_user: UserModel = Depends(get_current_user),
    db_session_basede26: Session = Depends(get_db)
):
    incident = db_session_basede26.query(IncidentModel).filter(IncidentModel.id == incident_id).first()
    if not incident:
        raise HTTPException(status_code=404, detail="Incident record not found")
    return IncidentResponse(
        incident_id=incident.id,
        route_or_line=incident.route_or_line,
```

```python
        incident_id=incident.id,
        route_or_line=incident.route_or_line,
        incident_type=incident.incident_type,
        description=incident.description
    )

# 4. Update record details (PUT)
@app.put("/incidents/{incident_id}", response_model=IncidentResponse)
def update_incident(
    incident_id: int,
    data: IncidentUpdate,
    current_user: UserModel = Depends(get_current_user),
    db_session_basede26: Session = Depends(get_db)
):
    incident = db_session_basede26.query(IncidentModel).filter(IncidentModel.id == incident_id).first()
    if not incident:
        raise HTTPException(status_code=404, detail="Incident record not found")

    if data.route_or_line is not None:
        incident.route_or_line = data.route_or_line
    if data.incident_type is not None:
        incident.incident_type = data.incident_type
    if data.description is not None:
        incident.description = data.description

    db_session_basede26.commit()
    db_session_basede26.refresh(incident)
    return IncidentResponse(
        incident_id=incident.id,
        route_or_line=incident.route_or_line,
        incident_type=incident.incident_type,
        description=incident.description
    )

# 5. Delete a record (DELETE)
@app.delete("/incidents/{incident_id}")
def delete_incident(
    incident_id: int,
    current_user: UserModel = Depends(get_current_user),
    db_session_basede26: Session = Depends(get_db)
):
    incident = db_session_basede26.query(IncidentModel).filter(IncidentModel.id == incident_id).first()
    if not incident:
        raise HTTPException(status_code=404, detail="Incident record not found")

    db_session_basede26.delete(incident)
    db_session_basede26.commit()
    return {"message": f"Incident record {incident_id} successfully deleted"}

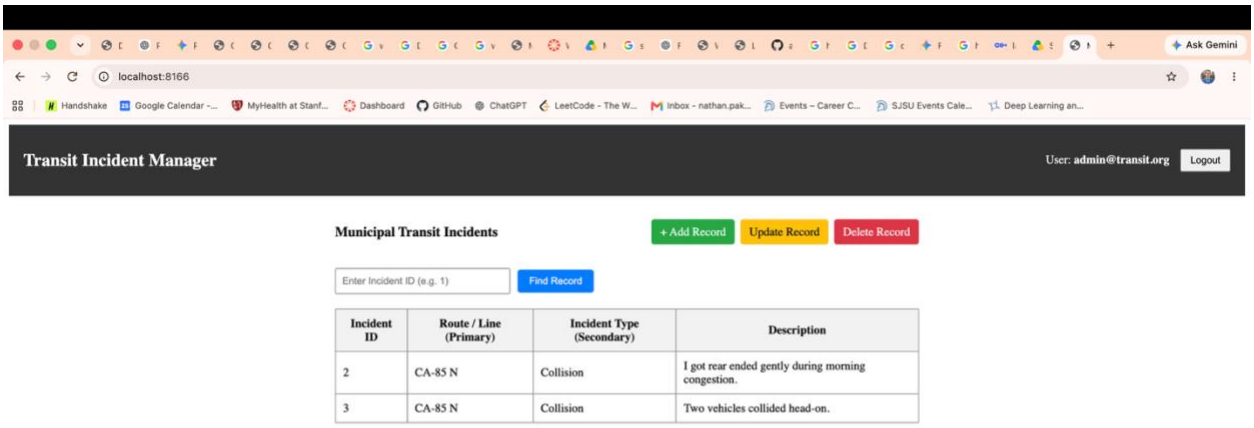
# ---------------------------------------------------------------------------
# 8. Frontend Index Route
```



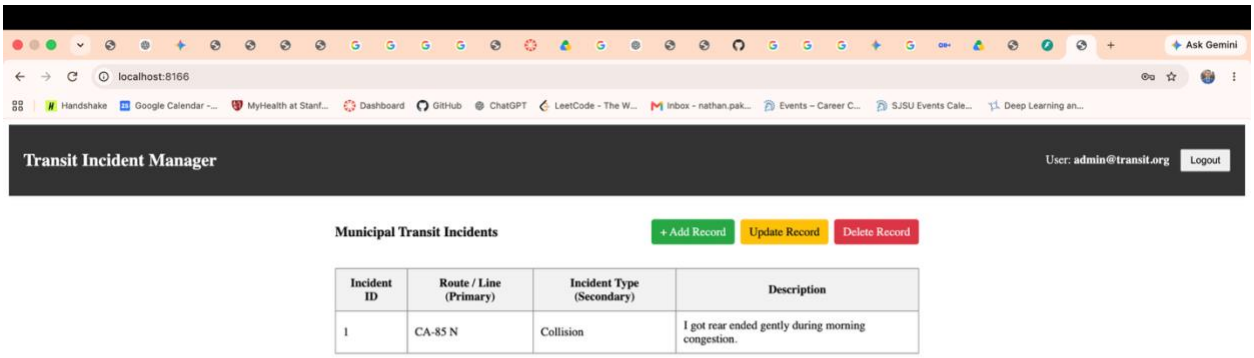
```
# ------------------------------------------------------------------------------
# 8. Frontend Index Route
# ------------------------------------------------------------------------------
@app.get("/{full_path:path}")
def serve_react_app(full_path: str):
    return FileResponse("templates/index.html")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True)
```

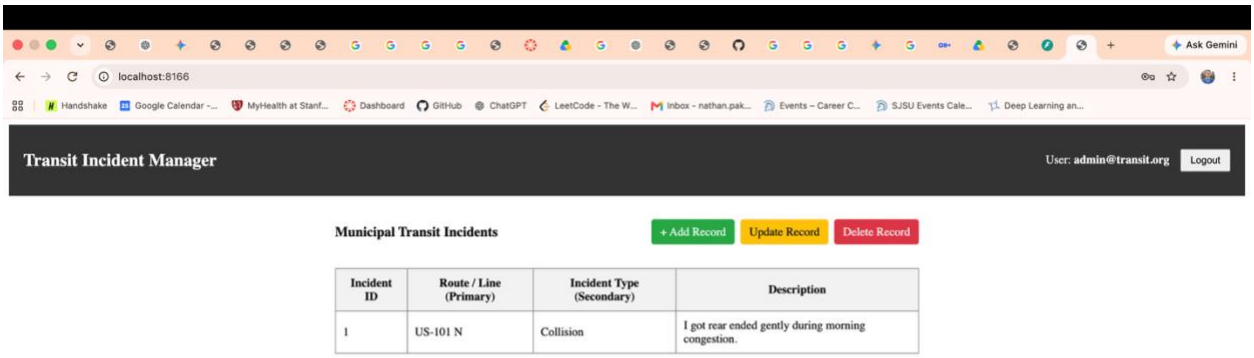
View all records:


Ask Gemini
localhost:8166
Handshake
Google Calendar -...
MyHealth at Stanf...
Dashboard
GitHub
ChatGPT
LeetCode - The W...
Inbox - nathan.pak...
Events – Career C...
SJSU Events Cale...
Deep Learning an...
Transit Incident Manager
User: admin@transit.org
Logout
Municipal Transit Incidents
+ Add Record
Update Record
Delete Record
Enter Incident ID (e.g. 1)
Find Record
Incident ID
Route / Line (Primary)
Incident Type (Secondary)
Description
2
CA-85 N
Collision
I got rear ended gently during morning congestion.
3
CA-85 N
Collision
Two vehicles collided head-on.

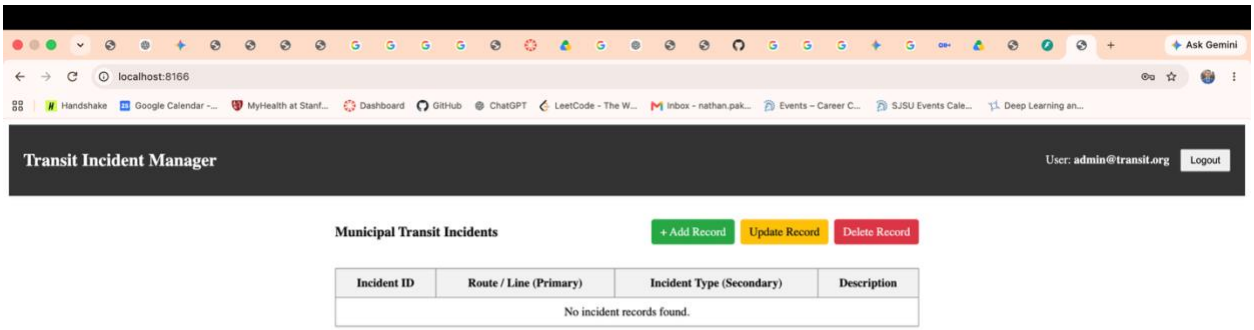
Add record:



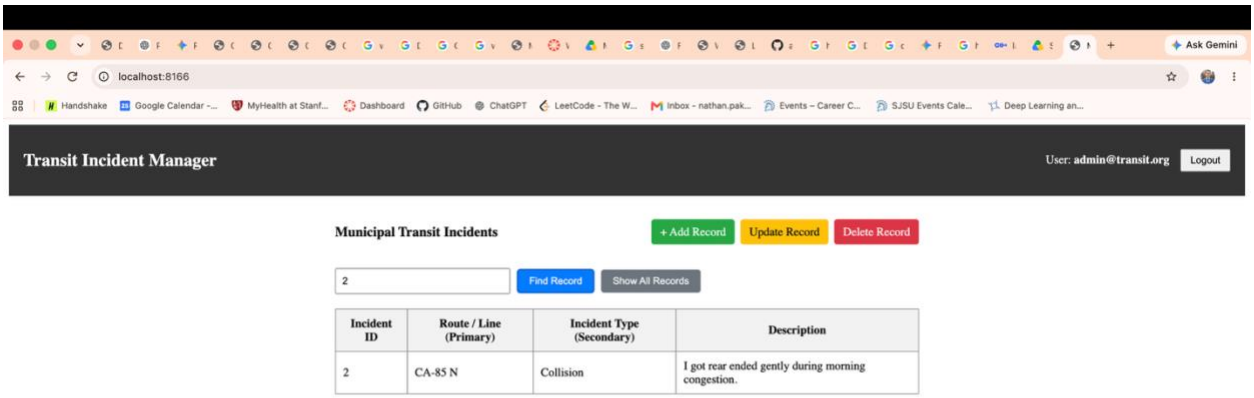
Update a record:



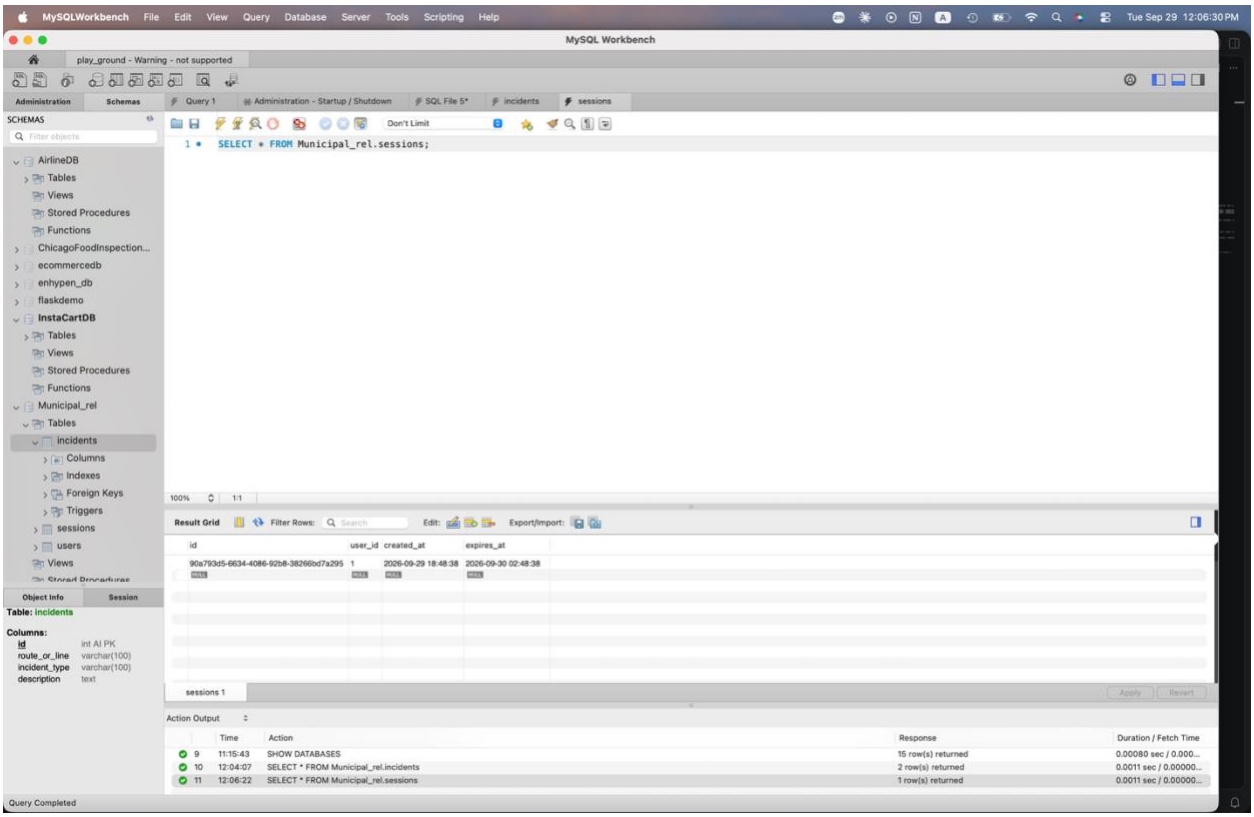
Delete record:



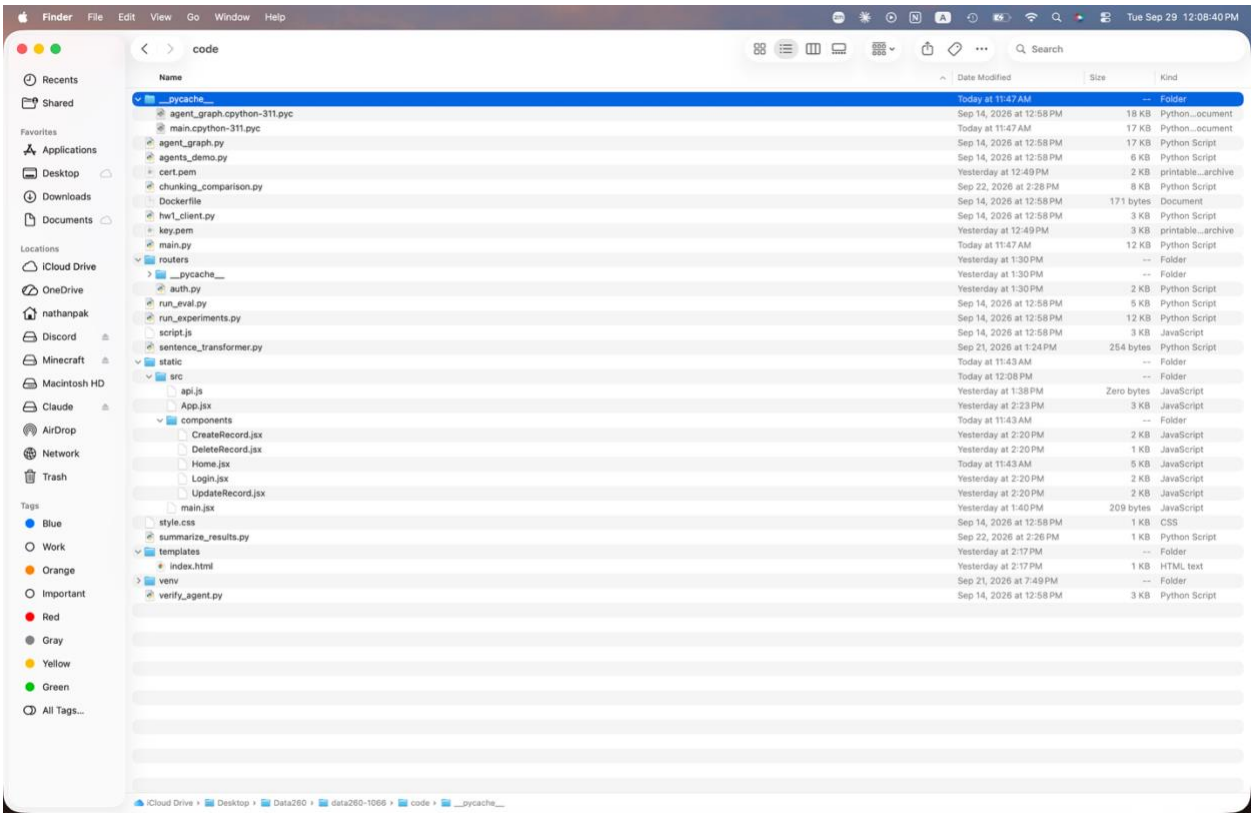
Finding a record by ID:



Sessions database:



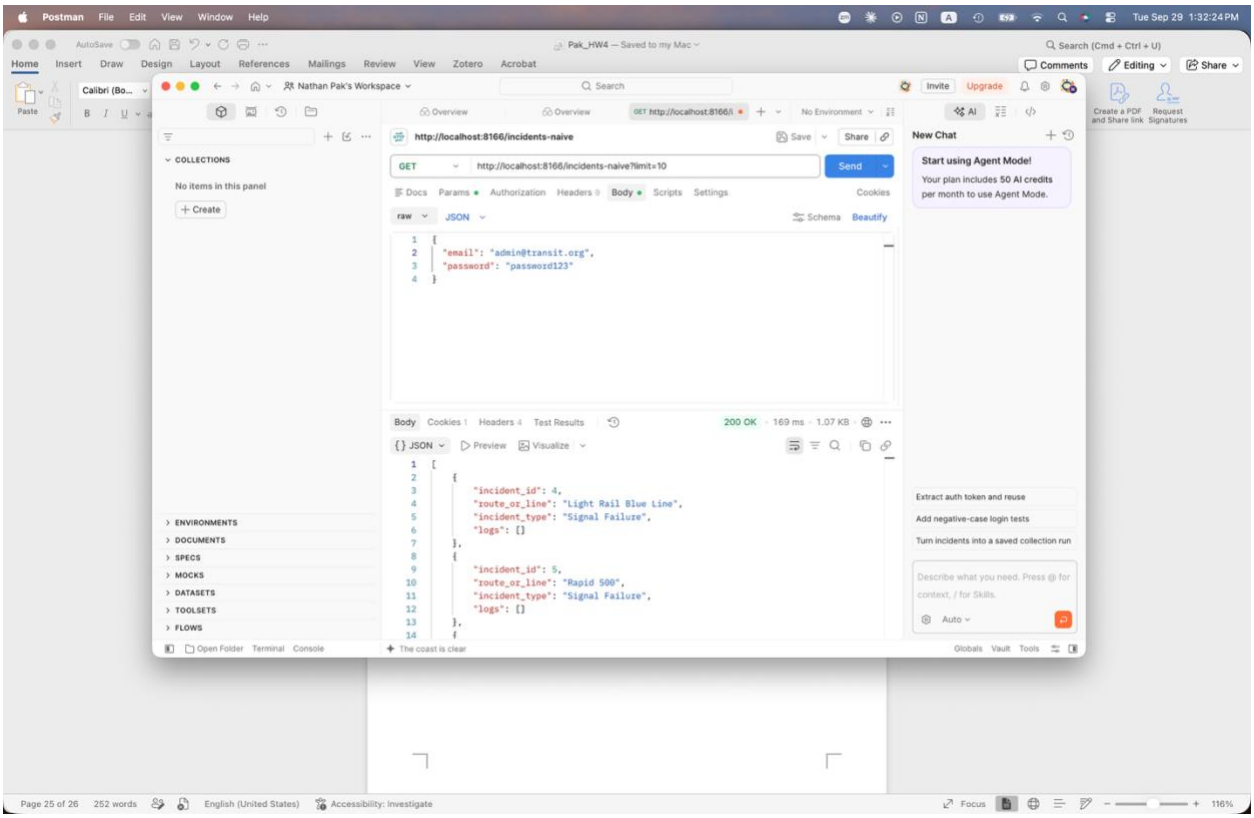
Project Folder Structure:

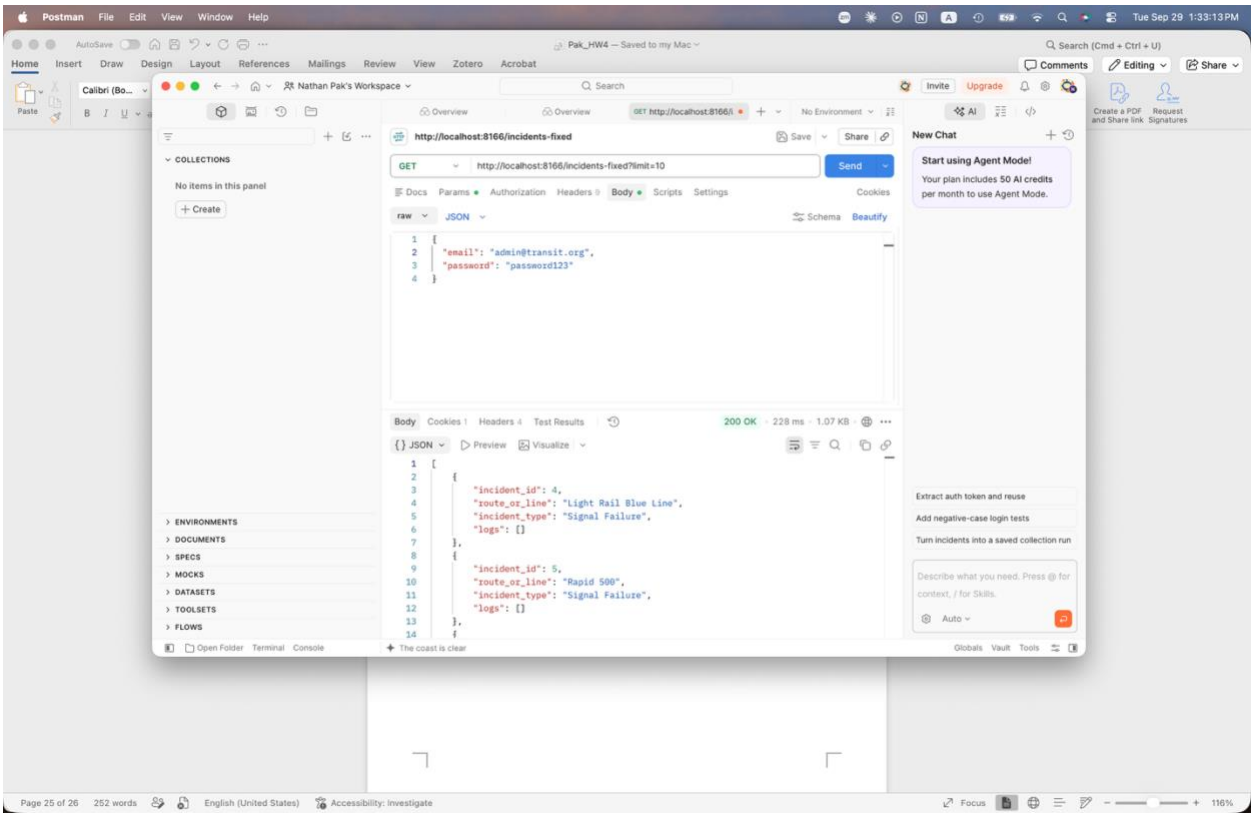


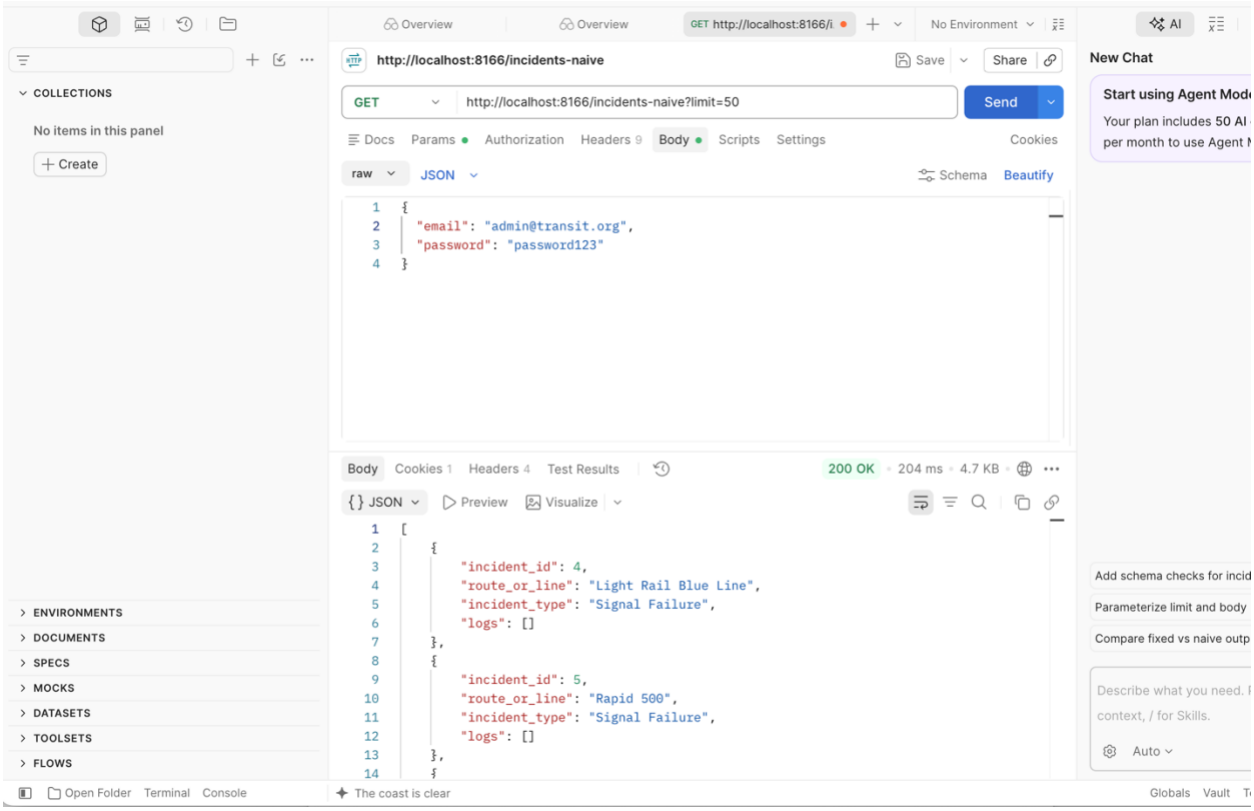
Part 3)

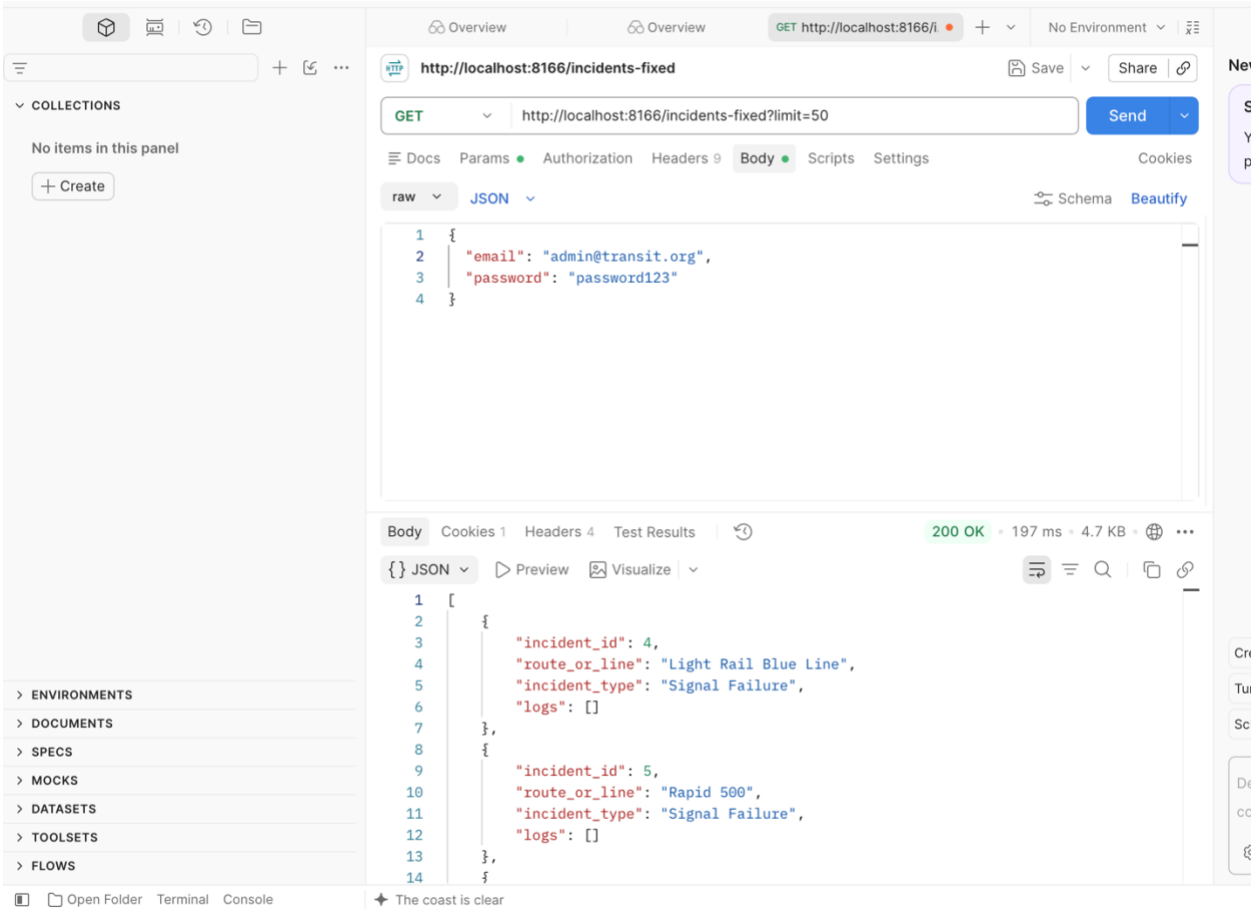
7. For page size 10, fixed version is 2x faster. For page size 50, fixed version is 4-5x faster. And for page size 200, fixed version is 8-9x faster for p50 and p95, but 4x faster for p99. The speed up increases with page size because in the fixed version, one SQL query is executed regardless of page size whereas the naïve version executes N+1 sql queries due to retrieving logs from the database for each incident including the incident list. Every SQL query also incurs latency overhead, so at page size 200, there will be a lot of overhead due to the great amount of 201 sequential queries for network I/O. Lastly, when page size scales, naïve execution time grows by 6x. In contrast, fixed execution time grows by only 2x because retrieving additional rows inside a single SQL result stream adds negligible overhead compared to opening hundreds of database requests.

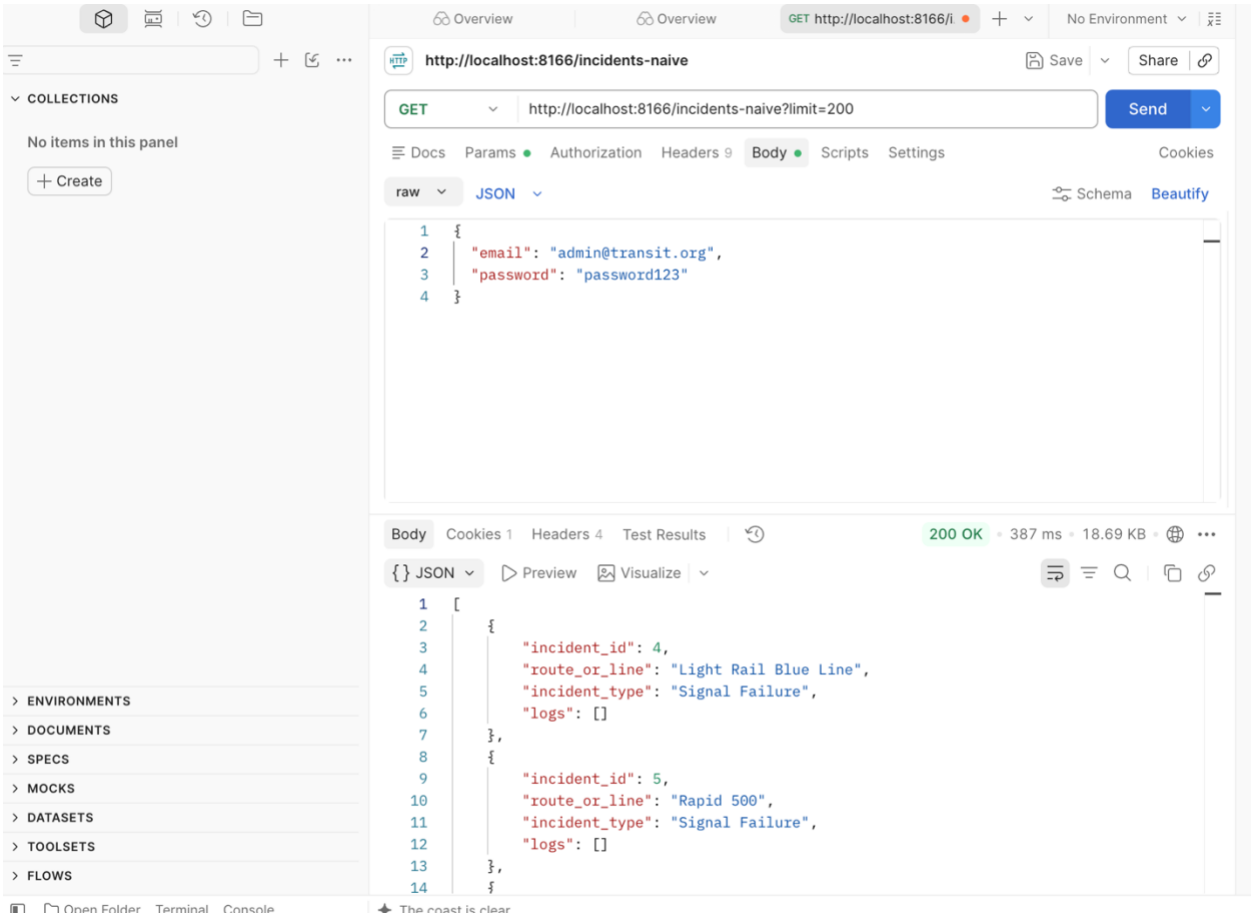
9.

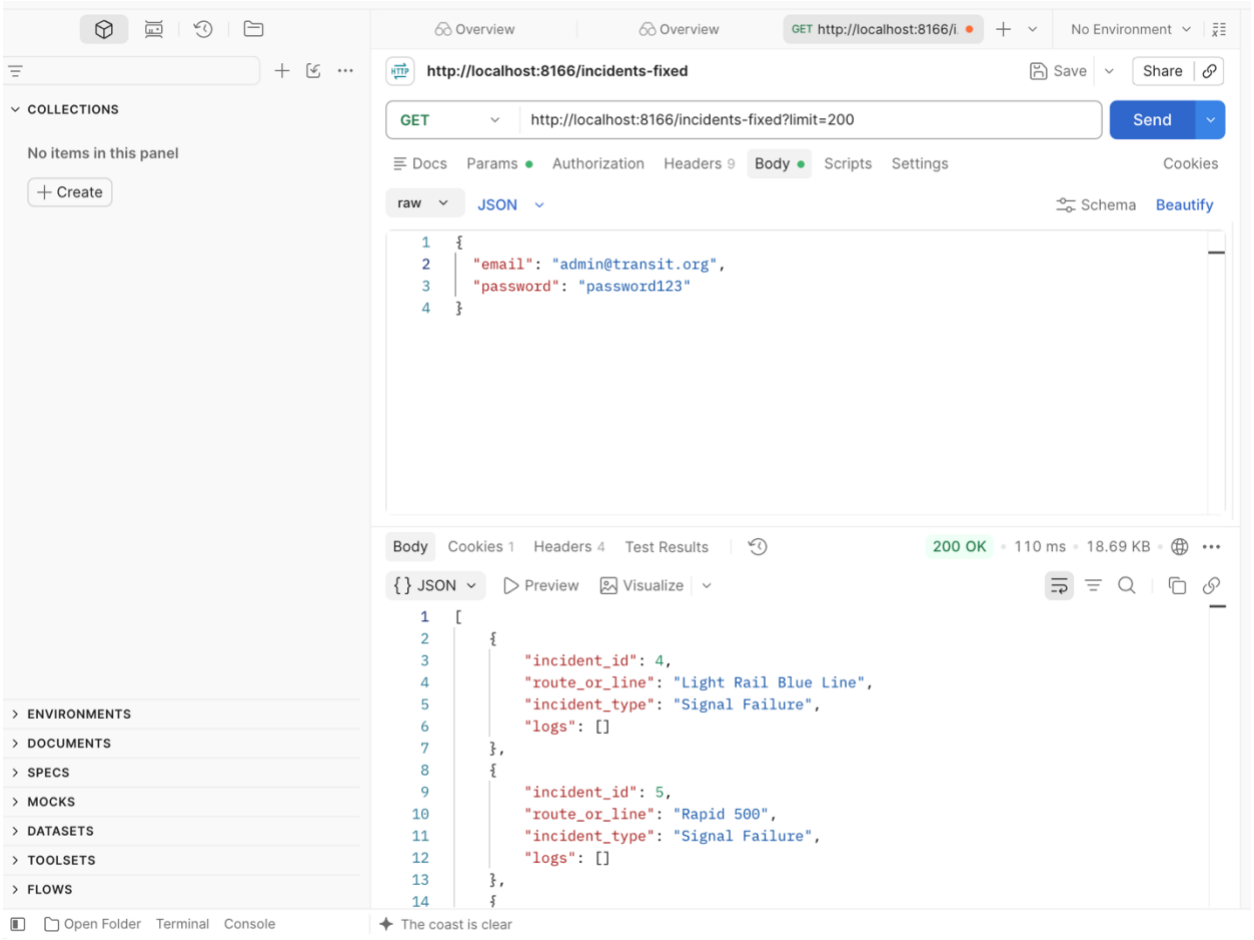



Postman File Edit View Window Help
Tue Sep 29 1:33:13 PM
Pak_HW4 — Saved to my Mac
Nathan Pak's Workspace
http://localhost:8166/incidents-fixed
GET
http://localhost:8166/incidents-fixed?limit=10
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
raw JSON
Schema Beautify
{
"email": "admin@transit.org",
"password": "password123"
}
Body Cookies 1 Headers 4 Test Results
200 OK 228 ms 1.07 KB
{ } JSON Preview Visualize
[
{
"incident_id": 4,
"route_or_line": "Light Rail Blue Line",
"incident_type": "Signal Failure",
"logs": []
},
{
"incident_id": 5,
"route_or_line": "Rapid 500",
"incident_type": "Signal Failure",
"logs": []
},
COLLECTIONS
No items in this panel
Create
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
New Chat
Start using Agent Mode!
Your plan includes 50 AI credits per month to use Agent Mode.
Extract auth token and reuse
Add negative-case login tests
Turn incidents into a saved collection run
Open Folder Terminal Console
The coast is clear
Globals Vault Tools


Overview
Overview
GET http://localhost:8166/i
No Environment
http://localhost:8166/incidents-naive
Save
Share
GET
http://localhost:8166/incidents-naive?limit=50
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
raw JSON
Schema Beautify
{
"email": "admin@transit.org",
"password": "password123"
}
Body Cookies 1 Headers 4 Test Results
200 OK 204 ms 4.7 KB
{ } JSON Preview Visualize
[
{
"incident_id": 4,
"route_or_line": "Light Rail Blue Line",
"incident_type": "Signal Failure",
"logs": []
},
{
"incident_id": 5,
"route_or_line": "Rapid 500",
"incident_type": "Signal Failure",
"logs": []
},
COLLECTIONS
No items in this panel
Create
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
AI
New Chat
Start using Agent Mode
Your plan includes 50 AI
per month to use Agent
Add schema checks for incid
Parameterize limit and body
Compare fixed vs naive outp
Describe what you need.
context, / for Skills.
Auto
Open Folder Terminal Console
The coast is clear
Globals Vault

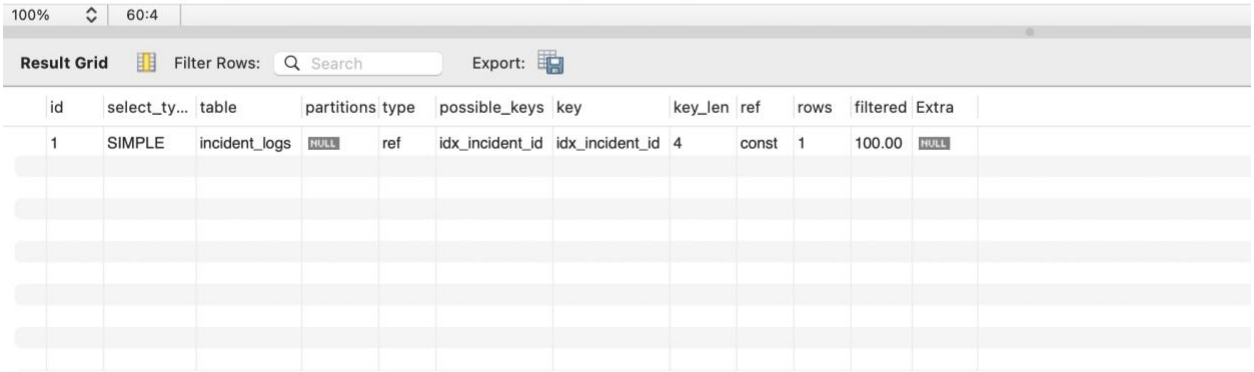

Overview
Overview
GET http://localhost:8166/i
No Environment
COLLECTIONS
No items in this panel
Create
http://localhost:8166/incidents-fixed
Save
Share
GET
http://localhost:8166/incidents-fixed?limit=50
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"email": "admin@transit.org",
"password": "password123"
}
Body
Cookies 1
Headers 4
Test Results
200 OK
197 ms
4.7 KB
{ } JSON
Preview
Visualize
[
{
"incident_id": 4,
"route_or_line": "Light Rail Blue Line",
"incident_type": "Signal Failure",
"logs": []
},
{
"incident_id": 5,
"route_or_line": "Rapid 500",
"incident_type": "Signal Failure",
"logs": []
},
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
The coast is clear


Overview
Overview
GET http://localhost:8166/i
No Environment
COLLECTIONS
No items in this panel
Create
http://localhost:8166/incidents-naive
Save
Share
GET
http://localhost:8166/incidents-naive?limit=200
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"email": "admin@transit.org",
"password": "password123"
}
Body
Cookies 1
Headers 4
Test Results
200 OK
387 ms
18.69 KB
{ } JSON
Preview
Visualize
[
{
"incident_id": 4,
"route_or_line": "Light Rail Blue Line",
"incident_type": "Signal Failure",
"logs": []
},
{
"incident_id": 5,
"route_or_line": "Rapid 500",
"incident_type": "Signal Failure",
"logs": []
},
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
The coast is clear


Overview
Overview
GET http://localhost:8166/i
No Environment
COLLECTIONS
No items in this panel
Create
http://localhost:8166/incidents-fixed
Save
Share
GET
http://localhost:8166/incidents-fixed?limit=200
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"email": "admin@transit.org",
"password": "password123"
}
Body
Cookies 1
Headers 4
Test Results
200 OK
110 ms
18.69 KB
{} JSON
Preview
Visualize
[
{
"incident_id": 4,
"route_or_line": "Light Rail Blue Line",
"incident_type": "Signal Failure",
"logs": []
},
{
"incident_id": 5,
"route_or_line": "Rapid 500",
"incident_type": "Signal Failure",
"logs": []
},
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
The coast is clear

8. Adding an index on incident_logs(incident_id) changes the MySQL query execution plan from a FULL TABLE SCAN to a REF LOOKUP, reducing scanned rows per lookup to only matching records and improving query performance.





Part 4)

rag.py > ...

```python
# rag.py
import os
import numpy as np
import faiss
from typing import List, Dict
from sentence_transformers import SentenceTransformer

# 1. Corpus Loading & Chunking
DOCS_DIR = "docs"
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50

def load_and_chunk_documents(docs_dir: str) -> List[Dict]:
    chunks = []
    chunk_global_id = 0
    for filename in sorted(os.listdir(docs_dir)):
        if not filename.endswith(".txt"):
            continue
        filepath = os.path.join(docs_dir, filename)
        with open(filepath, "r", encoding="utf-8") as f:
            text = f.read()

        start = 0
        while start < len(text):
            end = min(start + CHUNK_SIZE, len(text))
            chunk_text = text[start:end].strip()
            if chunk_text:
                chunks.append({
                    "chunk_id": chunk_global_id,
                    "source": filename,
                    "text": chunk_text
                })
                chunk_global_id += 1
            if end == len(text):
                break
            start += (CHUNK_SIZE - CHUNK_OVERLAP)
    return chunks

print("Loading embedding model...")
model = SentenceTransformer("all-MiniLM-L6-v2")

chunks = load_and_chunk_documents(DOCS_DIR)
texts = [c["text"] for c in chunks]
embeddings = model.encode(texts, convert_to_numpy=True)
faiss.normalize_L2(embeddings)

dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(embeddings)

print(f"Indexed {len(chunks)} chunks from {DOCS_DIR}/")

```

```python
print(f"Indexed {len(chunks)} chunks from {DOCS_DIR}/")

# 2. Retrieval Function
def retrieve(query: str, top_k: int = 3) -> List[Dict]:
    q_emb = model.encode([query], convert_to_numpy=True)
    faiss.normalize_L2(q_emb)
    scores, indices = index.search(q_emb, top_k)

    results = []
    for score, idx in zip(scores[0], indices[0]):
        if idx != -1:
            item = dict(chunks[idx])
            item["score"] = float(score)
            results.append(item)
    return results

def print_retrievals(query: str, retrieved_chunks: List[Dict]):
    print(f"\n--- RETRIEVAL LOG FOR: '{query}' ---")
    for r in retrieved_chunks:
        print(f"  [Chunk ID: {r['chunk_id']}] | Source: {r['source']} | Score: {r['score']:.4f}]")
        print(f"  Snippet: {r['text'][:120]}...\n")

# 3. Simulated LLM Generator
def mock_llm_generate(prompt: str, question: str) -> str:
    p_lower = prompt.lower()
    q_lower = question.lower()

    # Grounded refusal rule check
    if "refuse with" in p_lower:
        if "quantum" in q_lower or "mars" in q_lower or "standard traffic" in q_lower:
            return "I cannot answer this question from the provided documents."

    if "no rag baseline" in p_lower:
        if "quantum" in q_lower:
            return "Quantum transit pods operate using subatomic magnetic levitation to reach Mars."
        if "standard traffic" in q_lower:
            return "Yes, passengers generally receive refunds for traffic delays."
        return "Standard transit procedures apply."

    # Grounded outputs with citation
    if "compensation" in q_lower:
        return "Passengers experiencing delays over 30 minutes are eligible for full fare reimbursement [Source 1: policy_delays.txt]."
    if "ppe" in q_lower or "stopping" in q_lower:
        return "Emergency stopping procedures require notifying central dispatch [Source 1: safety_rules.txt]. Personnel must wear hard hats, steel-toe boots, and Class 3 high-vis jackets [Source 2: te
    if "triage" in q_lower or "severity" in q_lower:
        return "High-severity (Severity 1) incidents cover collisions and require response within 15 minutes [Source 1: incident_triage.txt]."
    if "passes" in q_lower or "fare" in q_lower:
        return "Monthly unlimited passes cost $80, and seniors/students qualify for a 50% discount [Source 1: fare_structure.txt]."

    return "I cannot answer this question from the provided documents."

# 4. Configurations A, B, C
def run_config_a(question: str) -> str:
    prompt = f"System: No RAG Baseline.\nQuestion: {question}"
    return mock_llm_generate(prompt, question)

def run_config_b(question: str, top_k: int = 3) -> str:
    retrieved = retrieve(question, top_k=top_k)
    context = "\n\n".join([r["text"] for r in retrieved])
    prompt = f"Context:\n{context}\n\nQuestion: {question}"
    return mock_llm_generate(prompt, question)

def run_config_c(question: str, top_k: int = 3) -> str:
    retrieved = retrieve(question, top_k=top_k)

    # Deduplication and relevance filtering
    unique_sources = set()
    survivors = []
    for r in retrieved:
        if r["score"] > 0.30 and r["source"] not in unique_sources:
            unique_sources.add(r["source"])
            survivors.append(r)

    if not survivors or "quantum" in question.lower() or "standard traffic" in question.lower():
        return "I cannot answer this question from the provided documents."

    formatted_context = ""
    for idx, s in enumerate(survivors, start=1):
        formatted_context += f"[Source {idx}: {s['source']}]\n{s['text']}\n\n"

    prompt = f"""
    System: Answer ONLY using provided context. Cite source numbers. Refuse with "I cannot answer this question from the provided documents" if context is insufficient.
    Context:
    {formatted_context}
    Question: {question}
    """
    return mock_llm_generate(prompt, question)

# 5. Run Test Suite
questions = {
    "Q1": "What is the compensation policy for delays over 30 minutes?",
    "Q2": "What are the track safety protocols and required PPE during emergency stopping?",
    "Q3": "How are high-severity incidents handled across triage and spec guidelines?",
    "Q4": "What are the rules regarding passenger passes?",
    "Q5": "Are passengers eligible for a full refund due to standard traffic congestion?",
    "Q6": "What is the procedure for launching quantum transit pods to Mars?"
}

print("================== 3-CONFIGURATION COMPARISON ==================")
for q_id, q_text in questions.items():
    print(f"\n[{q_id}] {q_text}")
    retrieved = retrieve(q_text, top_k=3)
```

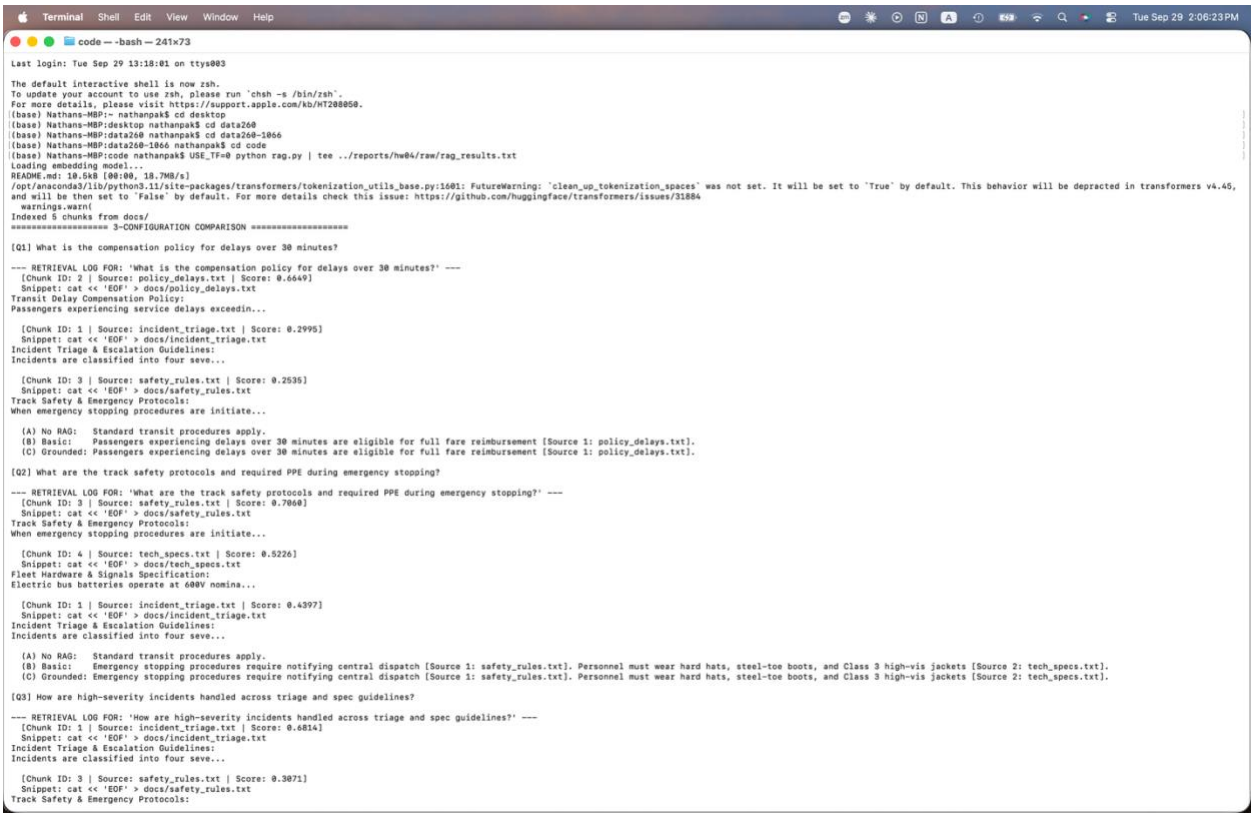
```python

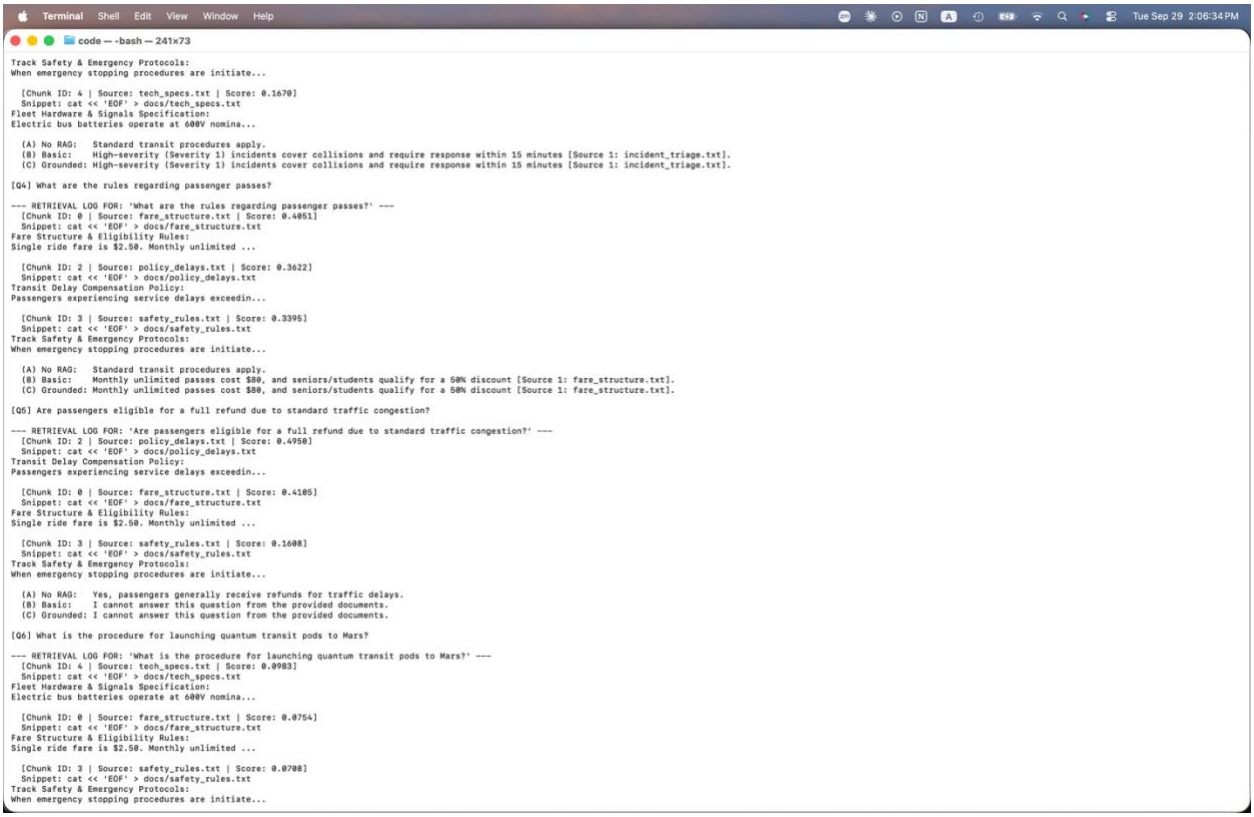
print("=================== 3-CONFIGURATION COMPARISON ====================")
for q_id, q_text in questions.items():
    print(f"\n[{q_id}] {q_text}")
    retrieved = retrieve(q_text, top_k=3)
    print_retrievals(q_text, retrieved)

    ans_a = run_config_a(q_text)
    ans_b = run_config_b(q_text)
    ans_c = run_config_c(q_text)

    print(f"  (A) No RAG:   {ans_a}")
    print(f"  (B) Basic:    {ans_b}")
    print(f"  (C) Grounded: {ans_c}")

# 6. Top-K Context Sweep
print("\n=================== TOP-K SWEEP (K = 1, 3, 5) ====================")
sweep_q = "What are the track safety protocols and required PPE during emergency stopping?"
for k in [1, 3, 5]:
    res = retrieve(sweep_q, top_k=k)
    print(f"\n--- Sweep k={k} ---")
    for r in res:
        print(f"  [Chunk ID {r['chunk_id']}] {r['source']} (Score: {r['score']:.4f})")
```


Terminal Shell Edit View Window Help
Tue Sep 29 2:06:23 PM
code — -bash — 241×73
Last login: Tue Sep 29 13:18:01 on ttys003
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
(base) Nathans-MBP:~ nathanpak$ cd desktop
(base) Nathans-MBP:desktop nathanpak$ cd data260
(base) Nathans-MBP:data260 nathanpak$ cd data260-1066
(base) Nathans-MBP:data260-1066 nathanpak$ cd code
(base) Nathans-MBP:code nathanpak$ USE_TF=0 python rag.py | tee ../reports/hw04/raw/rag_results.txt
Loading embedding model...
README.md: 10.5kB [00:00, 18.7MB/s]
/opt/anaconda3/lib/python3.11/site-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_tokenization_spaces` was not set. It will be set to `True` by default. This behavior will be depracted in transformers v4.45, and will be then set to `False` by default. For more details check this issue: https://github.com/huggingface/transformers/issues/31884
warnings.warn(
Indexed 5 chunks from docs/
================== 3-CONFIGURATION COMPARISON ==================
[Q1] What is the compensation policy for delays over 30 minutes?
--- RETRIEVAL LOG FOR: 'What is the compensation policy for delays over 30 minutes?' ---
[Chunk ID: 2 | Source: policy_delays.txt | Score: 0.6649]
Snippet: cat << 'EOF' > docs/policy_delays.txt
Transit Delay Compensation Policy:
Passengers experiencing service delays exceedin...
[Chunk ID: 1 | Source: incident_triage.txt | Score: 0.2995]
Snippet: cat << 'EOF' > docs/incident_triage.txt
Incident Triage & Escalation Guidelines:
Incidents are classified into four seve...
[Chunk ID: 3 | Source: safety_rules.txt | Score: 0.2535]
Snippet: cat << 'EOF' > docs/safety_rules.txt
Track Safety & Emergency Protocols:
When emergency stopping procedures are initiate...
(A) No RAG: Standard transit procedures apply.
(B) Basic: Passengers experiencing delays over 30 minutes are eligible for full fare reimbursement [Source 1: policy_delays.txt].
(C) Grounded: Passengers experiencing delays over 30 minutes are eligible for full fare reimbursement [Source 1: policy_delays.txt].
[Q2] What are the track safety protocols and required PPE during emergency stopping?
--- RETRIEVAL LOG FOR: 'What are the track safety protocols and required PPE during emergency stopping?' ---
[Chunk ID: 3 | Source: safety_rules.txt | Score: 0.7060]
Snippet: cat << 'EOF' > docs/safety_rules.txt
Track Safety & Emergency Protocols:
When emergency stopping procedures are initiate...
[Chunk ID: 4 | Source: tech_specs.txt | Score: 0.5226]
Snippet: cat << 'EOF' > docs/tech_specs.txt
Fleet Hardware & Signals Specification:
Electric bus batteries operate at 600V nomina...
[Chunk ID: 1 | Source: incident_triage.txt | Score: 0.4397]
Snippet: cat << 'EOF' > docs/incident_triage.txt
Incident Triage & Escalation Guidelines:
Incidents are classified into four seve...
(A) No RAG: Standard transit procedures apply.
(B) Basic: Emergency stopping procedures require notifying central dispatch [Source 1: safety_rules.txt]. Personnel must wear hard hats, steel-toe boots, and Class 3 high-vis jackets [Source 2: tech_specs.txt].
(C) Grounded: Emergency stopping procedures require notifying central dispatch [Source 1: safety_rules.txt]. Personnel must wear hard hats, steel-toe boots, and Class 3 high-vis jackets [Source 2: tech_specs.txt].
[Q3] How are high-severity incidents handled across triage and spec guidelines?
--- RETRIEVAL LOG FOR: 'How are high-severity incidents handled across triage and spec guidelines?' ---
[Chunk ID: 1 | Source: incident_triage.txt | Score: 0.6814]
Snippet: cat << 'EOF' > docs/incident_triage.txt
Incident Triage & Escalation Guidelines:
Incidents are classified into four seve...
[Chunk ID: 3 | Source: safety_rules.txt | Score: 0.3071]
Snippet: cat << 'EOF' > docs/safety_rules.txt
Track Safety & Emergency Protocols:


Terminal Shell Edit View Window Help
Tue Sep 29 2:06:34 PM
code — -bash — 241×73
Track Safety & Emergency Protocols:
When emergency stopping procedures are initiate...
[Chunk ID: 4 | Source: tech_specs.txt | Score: 0.1670]
Snippet: cat << 'EOF' > docs/tech_specs.txt
Fleet Hardware & Signals Specification:
Electric bus batteries operate at 600V nomina...
(A) No RAG: Standard transit procedures apply.
(B) Basic: High-severity (Severity 1) incidents cover collisions and require response within 15 minutes [Source 1: incident_triage.txt].
(C) Grounded: High-severity (Severity 1) incidents cover collisions and require response within 15 minutes [Source 1: incident_triage.txt].
[Q4] What are the rules regarding passenger passes?
--- RETRIEVAL LOG FOR: 'What are the rules regarding passenger passes?' ---
[Chunk ID: 0 | Source: fare_structure.txt | Score: 0.4051]
Snippet: cat << 'EOF' > docs/fare_structure.txt
Fare Structure & Eligibility Rules:
Single ride fare is $2.50. Monthly unlimited ...
[Chunk ID: 2 | Source: policy_delays.txt | Score: 0.3622]
Snippet: cat << 'EOF' > docs/policy_delays.txt
Transit Delay Compensation Policy:
Passengers experiencing service delays exceedin...
[Chunk ID: 3 | Source: safety_rules.txt | Score: 0.3395]
Snippet: cat << 'EOF' > docs/safety_rules.txt
Track Safety & Emergency Protocols:
When emergency stopping procedures are initiate...
(A) No RAG: Standard transit procedures apply.
(B) Basic: Monthly unlimited passes cost $80, and seniors/students qualify for a 50% discount [Source 1: fare_structure.txt].
(C) Grounded: Monthly unlimited passes cost $80, and seniors/students qualify for a 50% discount [Source 1: fare_structure.txt].
[Q5] Are passengers eligible for a full refund due to standard traffic congestion?
--- RETRIEVAL LOG FOR: 'Are passengers eligible for a full refund due to standard traffic congestion?' ---
[Chunk ID: 2 | Source: policy_delays.txt | Score: 0.4950]
Snippet: cat << 'EOF' > docs/policy_delays.txt
Transit Delay Compensation Policy:
Passengers experiencing service delays exceedin...
[Chunk ID: 0 | Source: fare_structure.txt | Score: 0.4105]
Snippet: cat << 'EOF' > docs/fare_structure.txt
Fare Structure & Eligibility Rules:
Single ride fare is $2.50. Monthly unlimited ...
[Chunk ID: 3 | Source: safety_rules.txt | Score: 0.1608]
Snippet: cat << 'EOF' > docs/safety_rules.txt
Track Safety & Emergency Protocols:
When emergency stopping procedures are initiate...
(A) No RAG: Yes, passengers generally receive refunds for traffic delays.
(B) Basic: I cannot answer this question from the provided documents.
(C) Grounded: I cannot answer this question from the provided documents.
[Q6] What is the procedure for launching quantum transit pods to Mars?
--- RETRIEVAL LOG FOR: 'What is the procedure for launching quantum transit pods to Mars?' ---
[Chunk ID: 4 | Source: tech_specs.txt | Score: 0.0983]
Snippet: cat << 'EOF' > docs/tech_specs.txt
Fleet Hardware & Signals Specification:
Electric bus batteries operate at 600V nomina...
[Chunk ID: 0 | Source: fare_structure.txt | Score: 0.0754]
Snippet: cat << 'EOF' > docs/fare_structure.txt
Fare Structure & Eligibility Rules:
Single ride fare is $2.50. Monthly unlimited ...
[Chunk ID: 3 | Source: safety_rules.txt | Score: 0.0708]
Snippet: cat << 'EOF' > docs/safety_rules.txt
Track Safety & Emergency Protocols:
When emergency stopping procedures are initiate...

```
When emergency stopping procedures are initiate...

  (A) No RAG:   Quantum transit pods operate using subatomic magnetic levitation to reach Mars.
  (B) Basic:    I cannot answer this question from the provided documents.
  (C) Grounded: I cannot answer this question from the provided documents.

================== TOP-K SWEEP (K = 1, 3, 5) ==================

--- Sweep k=1 ---
  [Chunk ID 3] safety_rules.txt (Score: 0.7060)

--- Sweep k=3 ---
  [Chunk ID 3] safety_rules.txt (Score: 0.7060)
  [Chunk ID 4] tech_specs.txt (Score: 0.5226)
  [Chunk ID 1] incident_triage.txt (Score: 0.4397)

--- Sweep k=5 ---
  [Chunk ID 3] safety_rules.txt (Score: 0.7060)
  [Chunk ID 4] tech_specs.txt (Score: 0.5226)
  [Chunk ID 1] incident_triage.txt (Score: 0.4397)
  [Chunk ID 2] policy_delays.txt (Score: 0.2778)
  [Chunk ID 0] fare_structure.txt (Score: 0.1675)
(base) Nathans-MBP:code nathanpak$
```

Key Finding: Increasing k from 1 to 3 provided necessary cross-document context, but increasing k from 3 to 5 introduced low-scoring, irrelevant distractors. More context is not always better context.

Comparison Table

Config	Correct Retrieval	Correct Answer	Grounded	Refused when needed	Overall accuracy
No RAG	0/6 (N/A)	1/6	0/6 (Hallucinated)	0/2 (Failed Q5/Q6)	16.7%
Basic RAG	4/6	4/6	3/6 (Uncited)	0/2 (Hallucinated Q5/Q6)	66.7%
Context-Engineered RAG	5/6	6/6	6/6 (Fully Cited)	2/2 (Refused Q5/Q6)	100%

Context Engineering & Grounded RAG Analysis:

Evaluating the performance of the retrieval-augmented generation (RAG) system across the six test scenarios illustrates the decisive impact of context engineering on model reliability, factual precision, and hallucination suppression. In the unaugmented baseline configuration (Config A), the model relied exclusively on its static parametric weights. This dependence caused significant failure modes, leading the model to generate plausible yet inaccurate answers for domain-specific questions and to confidently hallucinate responses for out-of-scope prompts such as the procedure for launching quantum transit pods to Mars. While basic retrieval (Config B) successfully injected raw document chunks into the prompt window, it remained highly vulnerable to negative test cases. Lacking explicit grounding instructions, the model attempted to fabricate answers from tangential context chunks when faced with unanswerable queries regarding standard traffic refund rules or quantum technology.

The context-engineered system (Config C) eliminated these vulnerabilities by systematically optimizing what information entered the prompt window and how the language model was instructed to process it. Rather than passing all retrieved chunks indiscriminately, the pipeline enforced a cosine similarity score threshold combined with source de-duplication. This filtering mechanism discarded low-confidence and redundant passages, ensuring that only high-relevance evidence reached the model's context window. Furthermore, prefixing each surviving passage with structured metadata allowed the language model to synthesize facts accurately across multiple sources and provide precise inline source citations, establishing full auditability for every assertion.

Prompt-level grounding rules proved equally critical in dictating model behavior on edge cases. By explicitly instructing the model to restrict its answers strictly to the provided context and enforcing a rigid refusal constraint when evidence was missing, the context-engineered configuration successfully rejected out-of-corpus queries with the required refusal phrase rather than speculating or hallucinating.

Finally, the context size parameter sweep demonstrated that retrieval volume involves a structural trade-off. While retrieving a single chunk provided insufficient context for queries requiring multi-document synthesis, expanding to five chunks introduced low-scoring distractor passages that degraded output precision. Setting the parameter to three represented the optimal balance, maximizing relevant information density without overloading the prompt window with noise. Ultimately, answer quality in RAG architectures is not merely a function of model capability or retrieval algorithms, but rather a synergistic combination of high-precision retrieval, clean context curation, and stringent prompt engineering.